HCMC University of Technology, VNU



Software Architecture

Chapter 4. Architecture Characteristics & Component-Based Thinking

Lecturer: Phat. T. Tran-Truong, Meng.

Email: phatttt@hcmut.edu.vn

Chapter 4. Architecture Characteristics



4.1. Architecture characteristics defined

4.2. Identifying architectural characteristics

4.3. Component-based thinking

4.1. Architecture characteristics defined



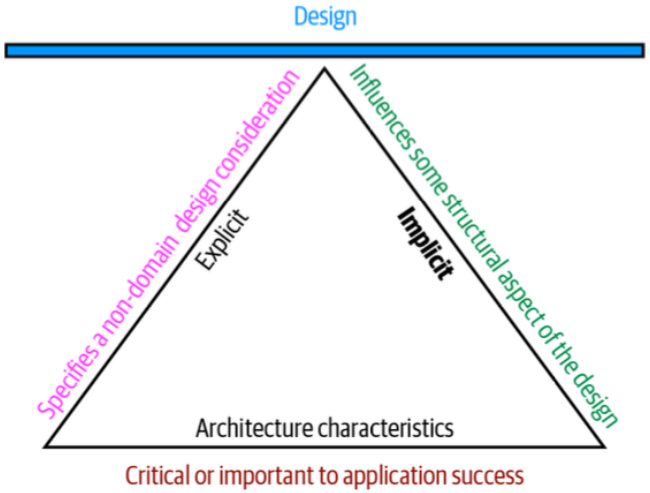
- Software requirements can be categorized as:
  - ***Functional requirements***: state what the system must do, how it must behave or react to run-time stimuli.
  - ***Non-functional requirements*** (aka: Quality attribute/characteristics requirements): constraints on the services or functions, often apply to the system as a whole rather than individual features or services.
    - Eg.: availability, performance, usability, security, etc.



4.1. Architecture characteristics defined



- An architecture characteristic meets three criteria:
  - Specifies a **non-domain design consideration** (explicit)
  - **Influences** some **structural aspect** of the design (implicit)
    - Eg: Third-party apps, in-application payment, etc.
  - Is **critical** or important to application **success**
    - Applications could support a huge number of architecture characteristics... but shouldn't.



4.1. Architecture characteristics defined



- Architectural Characteristics (Partially) Listed
  - Operational Architecture Characteristics

Term	Definition
Availability	How long the system will need to be available (if 24/7, steps need to be in place to allow the system to be up and running quickly in case of any failure).
Continuity	Disaster recovery capability.
Performance	Includes stress testing, peak analysis, analysis of the frequency of functions used, capacity required, and response times. Performance acceptance sometimes requires an exercise of its own, taking months to complete.
Recoverability	Business continuity requirements (e.g., in case of a disaster, how quickly is the system required to be on-line again?). This will affect the backup strategy and requirements for duplicated hardware.
Reliability/ safety	Assess if the system needs to be fail-safe, or if it is mission critical in a way that affects lives. If it fails, will it cost the company large sums of money?
Robustness	Ability to handle error and boundary conditions while running if the internet connection goes down or if there's a power outage or hardware failure.
Scalability	Ability for the system to perform and operate as the number of users or requests increases.

4.1. Architecture characteristics defined



- Architectural Characteristics (Partially) Listed
  - Structural Architecture Characteristics

Term	Definition
Configurability	Ability for the end users to easily change aspects of the software's configuration (through usable interfaces).
Extensibility	How important it is to plug new pieces of functionality in.
Installability	Ease of system installation on all necessary platforms.
Leverageability/ reuse	Ability to leverage common components across multiple products.
Localization	Support for multiple languages on entry/query screens in data fields; on reports, multibyte character requirements and units of measure or currencies.
Maintainability	How easy it is to apply changes and enhance the system?
Portability	Does the system need to run on more than one platform? (For example, does the frontend need to run against Oracle as well as SAP DB?
Supportability	What level of technical support is needed by the application? What level of logging and other facilities are required to debug errors in the system?
Upgradeability	Ability to easily/quickly upgrade from a previous version of this application/solution to a newer version on servers and clients.

4.1. Architecture characteristics defined



- Architectural Characteristics (Partially) Listed
  - Cross-cutting Architecture Characteristics

Term	Definition
Accessibility	Access to all your users, including those with disabilities like colorblindness or hearing loss.
Archivability	Will the data need to be archived or deleted after a period of time? (For example, customer accounts are to be deleted after three months or marked as obsolete and archived to a secondary database for future access.)
Authentication	Security requirements to ensure users are who they say they are.
Authorization	Security requirements to ensure users can access only certain functions within the application (by use case, subsystem, webpage, business rule, field level, etc.).
Legal	What legislative constraints is the system operating in (data protection, Sarbanes Oxley, GDPR, etc.)? What reservation rights does the company require? Any regulations regarding the way the application is to be built or deployed?
Privacy	Ability to hide transactions from internal company employees (encrypted transactions so even DBAs and network architects cannot see them).
Security	Does the data need to be encrypted in the database? Encrypted for network communication between internal systems? What type of authentication needs to be in place for remote user access?
Supportability	What level of technical support is needed by the application? What level of logging and other facilities are required to debug errors in the system?
Usability/ achievability	Level of training required for users to achieve their goals with the application/solution. Usability requirements need to be treated as seriously as any other architectural issue.

4.1. Architecture characteristics defined



- ISO have listed some of definitions:
  - Performance efficiency
  - Compatibility
  - Usability
  - Security
  - Reliability
  - Maintainability
  - Portability

4.1. Architecture characteristics defined



Performance efficiency

- Measure of the performance relative to the amount of resources used under known conditions.
- Main sub-characteristics:
  - **Time behavior** - measure of response, processing times, and/or throughput rates.
  - **Resource utilization** - amounts and types of resources used.
  - **Capacity** - degree to which the maximum established limits are exceeded.

4.1. Architecture characteristics defined



Compatibility

- Degree to which a product, system, or component can exchange information with other products, systems, or components and/or perform its required functions while sharing the same hardware or software environment.
- The main sub-characteristics:
  - **Coexistence** - can perform its required functions efficiently while sharing a common environment and resources with other products.
  - **Interoperability** - degree to which two or more systems can exchange and utilize information.

4.1. Architecture characteristics defined



Usability

- This software quality attribute helps define the **ease** to use for specific tasks on the system.
- The main sub-characteristics:
  - **Appropriateness recognizability** - users can recognize whether the software is appropriate for their needs.
  - **Learnability -** how easy users can learn how to use the software.
  - **User error protection** - protection against users making errors.
  - **Accessibility** - make the software available to people with the widest range of characteristics and capabilities.

4.1. Architecture characteristics defined



Security

- Degree the software protects information and data so that people or other products or systems have the degree of data access appropriate to their types and levels of authorization.
- The main sub-characteristics:
  - **Confidentiality** - data is accessible only to those authorized to have access.
  - **Integrity** - the software prevents unauthorized access to or modification of software or data.
  - **Nonrepudiation** - can actions or events be proven to have taken place.
  - **Accountability** - can user actions of a user be traced.
  - **Authenticity** - proving the identity of a user.

4.1. Architecture characteristics defined



Reliability

- The degree to which a software or its components performs **specific functions** under predefined conditions in a certain period of time.
- The main sub-characteristics:
  - **Maturity -** does the software meet the reliability needs under normal operation.
  - **Availability** – software is operational and accessible.
  - **Fault tolerance** - does the software operate as intended despite hardware or software faults.
  - **Recoverability** - can the software recover from failure by recovering any affected data and reestablish the desired state of the system.

4.1. Architecture characteristics defined



Maintainability

- Represents the degree of effectiveness and efficiency to which developers can modify the software to improve it, correct it, or adapt it to changes in environment and/or requirements.
- The main sub-characteristics:
  - **Modularity** - degree to which the software is composed of discrete components.
  - **Reusability** - degree to which developers can use an asset in more than one system or in building other assets.
  - **Analyzability** - how can gather concrete metrics about the software.
  - **Modifiability** - degree to which developers can modify the software without introducing defects or degrading existing product quality.
  - **Testability** - how can test the software.

4.1. Architecture characteristics defined



Portability

- Degree to which developers can transfer a system, product, or component from one hardware, software, or other operational or usage environment to another.
- The main sub-characteristics:
  - **Adaptability** - can adapt effectively and efficiently the software for different or evolving hardware, software, or other operational or usage environments.
  - **Installability** - can the software be installed and/or uninstalled in a specified environment),
  - **Replaceability** - how can replace the functionality with other software.

4.1. Architecture characteristics defined



Example

- Scenarios should be prioritized
- 2 variables from 3 values (Low/Medium/High)
  - How important it is for success (ranked by customer/BA)
  - How difficult it is to achieve (ranked by architect)

Ref	Quality Attribute	Scenario	Priority
1	Availability	When the database does not respond, the system should log the fault and respond with stale data during 3 seconds	High, High
2	Availability	A user searches for elements of type X and receives a list of Xs 99% of the time on average over the course of the year	High, Medium
3	Scalability	New servers can be added during the planned maintainance window (less than 7 hours)	Low, Low
4	Performance	A user sees search results within 5 seconds when the system is at an average load of 2 searches per second	High, High
5	Reliability	Updates to external elements of type X should be reflected on the application within 24 hours of the change	Low, Medium
	. . .	. . .	. . .

4.1. Architecture characteristics defined



Trade-offs and least worst architecture

- In software architecture: “Everything is a trade-off”.
  - Eg: if an architect wants to improve security, it will negatively impact performance: on-the-fly encryption, indirection for secrets hiding, etc.
- Never shoot for the best architecture, but rather the **least worst architecture**.

4.2. Identifying Architectural Characteristics



- **Identifying** the driving architectural **characteristics** is **one of** the **first steps** in **creating** an **architecture** or **determining** the **validity** of an existing architecture.
- Identifying the correct architectural characteristics ("-ilities") for a given problem or application requires an architect to not only **understand** the **domain problem**, but also **collaborate** with the problem **domain stakeholders** to **determine what** is truly **important** from a domain perspective.
- An architect **discovers** architecture **characteristics** in **at least three ways** by extracting from **domain concerns**, **requirements**, and **implicit domain knowledge**.
- It is critical for the architect to **collaborate with the developers, project manager, operations team**, and **other co-constructors** of the software system. **No architecture decision** should be made **isolated** from the implementation team.

4.2. Identifying Architectural Characteristics



- Extracting from **Domain Concerns**.
  - An architect **must be able to translate domain concerns to** identify the **right** architectural **characteristics**.
  - **Understanding** the **key domain goals** and **domain situation** allows an architect to **translate those domain concerns to "-ilities,"** which then forms the basis for correct and justifiable architecture decisions.
  - One tip when collaborating with domain stakeholders to define the driving architecture characteristics is to work hard to **keep** the **final list as short as possible**.
  - Supporting **too many** architecture **characteristics** leads to greater and greater **complexity** before the architect and developers have even started addressing the problem domain, the original motivation for writing the software.
  - Don't obsess over the number of characteristics, but rather the motivation to **keep design simple**.

4.2. Identifying Architectural Characteristics



- Extracting from **Domain Concerns**.
  - Architects and domain stakeholders speak different languages.
  - “Lost in translation” problem where the architect and domain stakeholder don’t understand each other.
  - There is usually a translation from domain concerns to architecture characteristics.

Domain concern	Architecture characteristics
Mergers and acquisitions	Interoperability, scalability, adaptability, extensibility
Time to market	Agility, testability, deployability
User satisfaction	Performance, availability, fault tolerance, testability, deployability, agility, security
Competitive advantage	Agility, testability, deployability, scalability, availability, fault tolerance
Time and budget	Simplicity, feasibility

Translation of domain concerns to architecture characteristics

4.2. Identifying Architectural Characteristics



- Extracting from **Requirements**.
  - **Some** architecture **characteristics** come from **explicit statements** in requirements documents.
  - **Others** come from **inherent domain knowledge** by architects, one of the many reasons that domain knowledge is always beneficial for architects.
  - For example, suppose an architect designs an application that handles class registration for university students. To make the math easy, assume that the school has **1,000 students** and **10 hours for registration**. Should an architect design a system assuming consistent scale, making the implicit assumption that the students during the **registration process** will **distribute themselves evenly over time**? Or, based on knowledge of university students habits and proclivities, should the architect design a system that can **handle all 1,000 students** attempting to register in the **last 10 minutes**?

4.2. Identifying Architectural Characteristics



- Extracting from **Implicit domain knowledge**.
  - Many architecture characteristics **aren't specified in requirements documents**, yet they make up an **important aspect** of the **design**.
  - For example, **Security** appears as an **implicit characteristic** in **every system**: no one wants to create insecure software. However, it may be prioritized depending on criticality, which illustrates the interlocking nature of our definition. An architect **considers security** as an architecture **characteristic if** it influences some **structural aspect** of the design and is **critical** or **important** to the application.

4.2. Identifying Architectural Characteristics



- **Architecture Katas**
  - Devised by **Ted Neward**, a clever method to **derive architecture characteristics** from domain-targeted descriptions.
  - The basic premise of the kata exercise provides architects with a **problem stated in domain terms** and **additional context** (things that might not appear in requirements yet impact design). Small teams work for **~25 minutes** on a design, then show results to the other groups and **vote** to come up with the best solution:
    - Problem description: ~10 mins
    - Identify architectural characteristics: ~15 mins

4.2. Identifying Architectural Characteristics



- **Architecture Katas**
  - Each katas has predefined sections:
    - ***Description***: The overall domain problem the system is trying to solve.
    - ***Users***: The expected number and/or types of users of the system.
    - ***Requirements***: Domain/domain-level requirements, as an architect might expect from domain users/domain experts.
    - ***Additional context***: Many of the considerations an architect must make aren't explicitly expressed in requirements but rather by implicit knowledge of the problem domain.
  - Team members ideally **get feedback** from the experienced architect about **missed trade-offs** and **alternative designs**.

4.2. Identifying Architectural Characteristics



- **Architecture Katas**

Your Architectural Kata is...

Going Going Gone!

An auction company wants to take their auctions online to a nationwide scale--customers choose the auction to participate in, wait until the auction begins, then bid during the live auction as if they were there in the room, with the auctioneer.

- **Users**: scale up to hundreds of participants (per auction), potentially up to thousands of participants, and as many simultaneous auctions as possible
- **Requirements**:
  - bidders can see a live video stream of the auction and see all bids as they occur
  - auctions must be as real-time as possible
  - both online and live bids must be received in the order in which they are placed
  - bidders register with credit card; system automatically charges card if bidder wins
  - participants must be tracked via a reputation index
- **Additional Context:**
  - auction company is expanding aggressively by merging with smaller competitors
  - if nationwide auction is a success, replicate the model overseas
  - budget is not constrained--this is a strategic direction
  - company just exited a lawsuit where they settled a suit alleging fraud

4.2. Identifying Architectural Characteristics



- **Architecture Katas**

Your Architectural Kata is...

Going Going Gone!

An auction company wants to take their auctions online to a nationwide scale--customers choose the auction to participate in, wait until the auction begins, then bid during the live auction as if they were there in the room, with the auctioneer.

- **Users**: scale up to hundreds of participants (per auction), potentially up to thousands of participants, and as many simultaneous auctions as possible
- **Requirements**:
  - bidders can see a live video stream of the auction and see all bids as they occur
  - auctions must be as real-time as possible
  - both online and live bids must be received in the order in which they are placed
  - bidders register with credit card; system automatically charges card if bidder wins
  - participants must be tracked via a reputation index
- **Additional Context:**
  - auction company is expanding aggressively by merging with smaller competitors
  - if nationwide auction is a success, replicate the model overseas
  - budget is not constrained--this is a strategic direction
  - company just exited a lawsuit where they settled a suit alleging fraud

availability reliability performance scalability elasticity concurrency (security)

4.2. Identifying Architectural Characteristics



- **Architecture Katas**

Architecture Characteristics Worksheet

System/Project: _____ Domain/Quantum: _____

Architect/Team: _____ Date: _____ Next Review: _____

Top 3	Driving Characteristics	Implicit Characteristics
X	availability	feasibility (cost/time)
	reliability	security
X	performance	maintainability
	scalability	observability
X	elasticity	
	security	
	concurrency	

Others Considered

_____

_____

_____

_____

Instructions

- Identify no more than 7 driving characteristics.
- Pick the top 3 characteristics (in any order).
- Implicit characteristics can become driving characteristics if they are *critical* concerns.
- Add additional characteristics identified that weren't deemed as important as the list of 7 to the *Others Considered* list.
- Definitions are on the following pages

Common Architecture Characteristics

performance	data integrity	deployability
responsiveness	data consistency	testability
availability	adaptability	abstraction
fault tolerance	extensibility	workflow
scalability	interoperability	configurability
elasticity	concurrency	recoverability

Composite Architecture Characteristics

agility → maintainability, testability, deployability

reliability → availability, testability, data integrity, data consistency, fault tolerance

a / b denotes characteristics that are related; some systems only need one of these, other systems may need both

Last updated March 2024 — Created by Mark Richards, DeveloperToArchitect.com

4.2. Identifying Architectural Characteristics



Exercise: Silicon Sandwiches

- **Description:** A national sandwich shop wants to enable online ordering (in addition to its current call-in service)
- **Users:** Thousands, perhaps one day millions
- **Requirements:**
  - ***ER1:*** Users will place their order, then be given a time to pick up their sandwich and directions to the shop (which must integrate with several external mapping services that include traffic information)
  - ***ER2:*** If the shop offers a delivery service, dispatch the driver with the sandwich to the user
  - ***ER3:*** Mobile-device accessibility
  - ***ER4:*** Offer national daily promotions/specials
  - ***ER5:*** Offer local daily promotions/specials
  - ***ER6:*** Accept payment online, in person, or upon delivery
- **Additional Context:**
  - ***ER7:*** Sandwich shops are franchised, each with a different owner
  - ***ER8:*** Parent company has near-future plans to expand overseas
  - ***ER9:*** Corporate goal is to hire inexpensive labor to maximize profit

4.2. Identifying Architectural Characteristics



Exercise: Silicon Sandwiches

Use the following sheet to identify characteristics from the problem description.

Architecture Characteristics Worksheet

System/Project: _____ Domain/Quantum: _____

Architect/Team: _____ Date: _____ Next Review: _____

Top 3	Driving Characteristics	Implicit Characteristics
☐	_____	feasibility (cost/time)
☐	_____	security
☐	_____	maintainability
☐	_____	observability
☐	_____	_____
☐	_____	
☐	_____	Others Considered
		_____
		_____
		_____
		_____

Instructions

- Identify no more than 7 driving characteristics.
- Pick the top 3 characteristics (in any order).
- Implicit characteristics can become driving characteristics if they are *critical* concerns.
- Add additional characteristics identified that weren't deemed as important as the list of 7 to the *Others Considered* list.
- Definitions are on the following pages

Common Architecture Characteristics

performance	data integrity	deployability
responsiveness	data consistency	testability
availability	adaptability	abstraction
fault tolerance	extensibility	workflow
scalability	interoperability	configurability
elasticity	concurrency	recoverability

Composite Architecture Characteristics

agility → maintainability, testability, deployability

reliability → availability, testability, data integrity, data consistency, fault tolerance

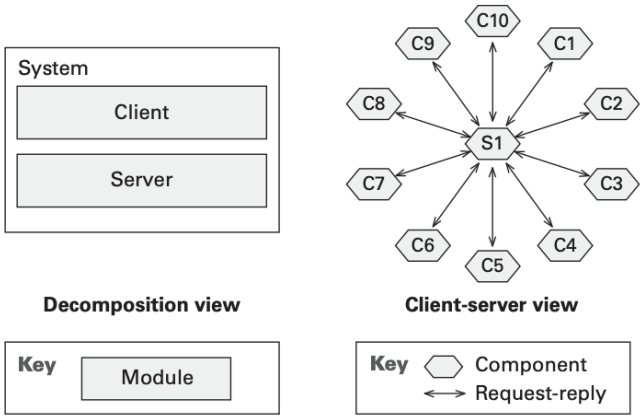
a / b denotes characteristics that are related; some systems only need one of these, other systems may need both

4.3. Component-based thinking

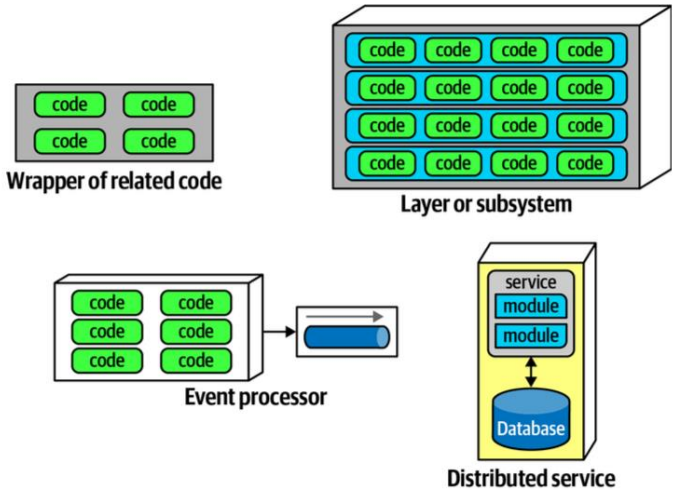


Component vs Module

- Module: a collection of related code; unit of implementation.
- Component: a runtime entity; unit of deployment. For examples:
  - **Library** – wrapper of code to run in the same memory address as the calling code and communicate via language function call mechanisms.
  - **Service** - wrapper of code to run in its own address space and communicates via low-level networking protocols like TCP/IP or higher-level formats like REST or message queues, forming standalone, deployable units in architectures like microservices.



A client-server system might consist of two modules but eleven components.

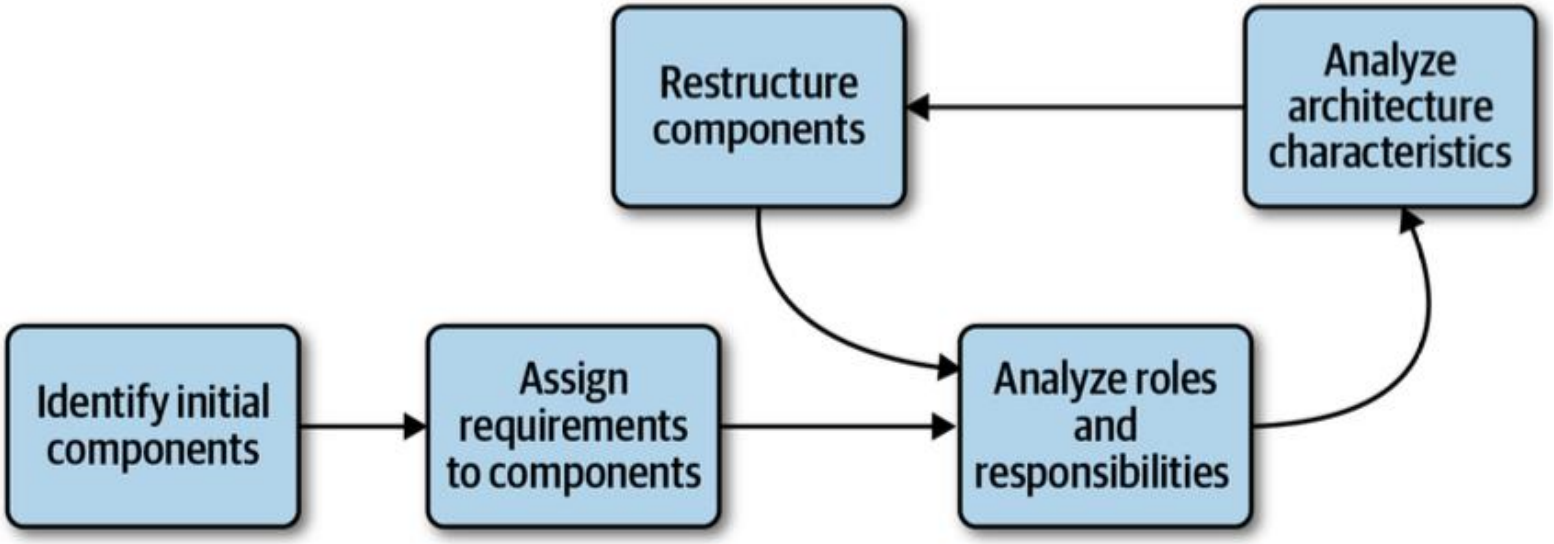


Different varieties of components

4.3. Component-based thinking



Component Identification Flow



Component identification cycle

4.3. Component-based thinking



Component Identification Flow

- **Identifying Initial Components:**
  - Determine what top-level components to begin with, based on what type of top-level partitioning chosen.
  - The likelihood of achieving a good design from this initial set of components is disparagingly small, which is why architects must iterate on component design to improve it.
- **Assign Requirements to Components:**
  - Align requirements (or user stories) to the initial components to see how well they fit.
  - This may entail creating new components, consolidating existing ones, or breaking components apart because they have too much responsibility.

4.3. Component-based thinking



Component Identification Flow

- **Analyze Roles and Responsibilities:**
  - Look at the roles and responsibilities elucidated during the requirements to make sure that the granularity matches.
  - Thinking about both roles and behaviors the application must support allows the architect to align the component and domain granularity.
- **Analyze Architecture Characteristics:**
  - When assigning requirements to components, the architect should also look at the architecture characteristics discovered earlier in order to think about how they might impact component division and granularity.
- **Restructure Components:**
  - Architects must continually iterate on their component design with developers.
  - an iterative approach to component design is key.

4.3. Component-based thinking



Component granularity & design

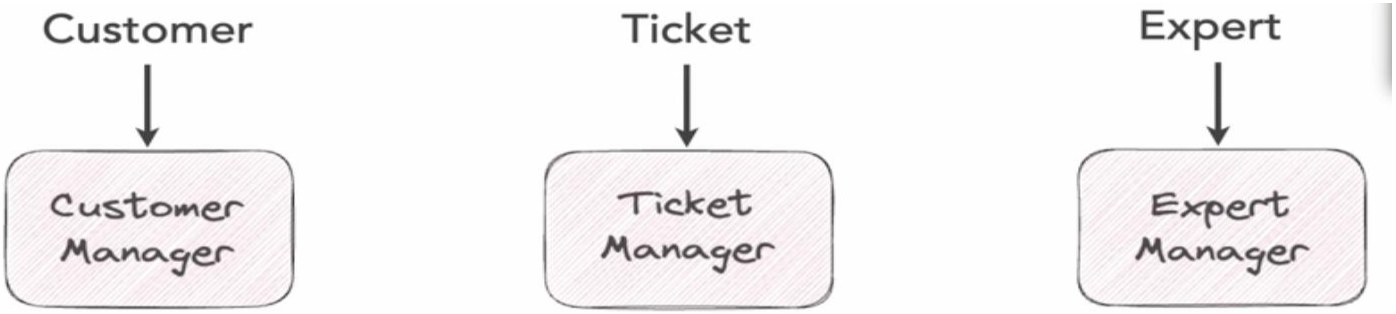
- Too fine-grained a component design leads to too much communication between components to achieve results.
- Too coarse-grained components encourage high internal coupling, which leads to difficulties in deployability and testability, as well as modularity-related negative side effects.
- No accepted “correct” way exists to design components. Rather, a wide variety of techniques exist, all with various trade-offs.

4.3. Component-based thinking



Discovering components

- **Entity trap anti-pattern**
- The architect has basically taken each entity identified in the requirements and made a Manager component based on that entity. This isn't an architecture; it's a component-relational mapping of a framework to a database.



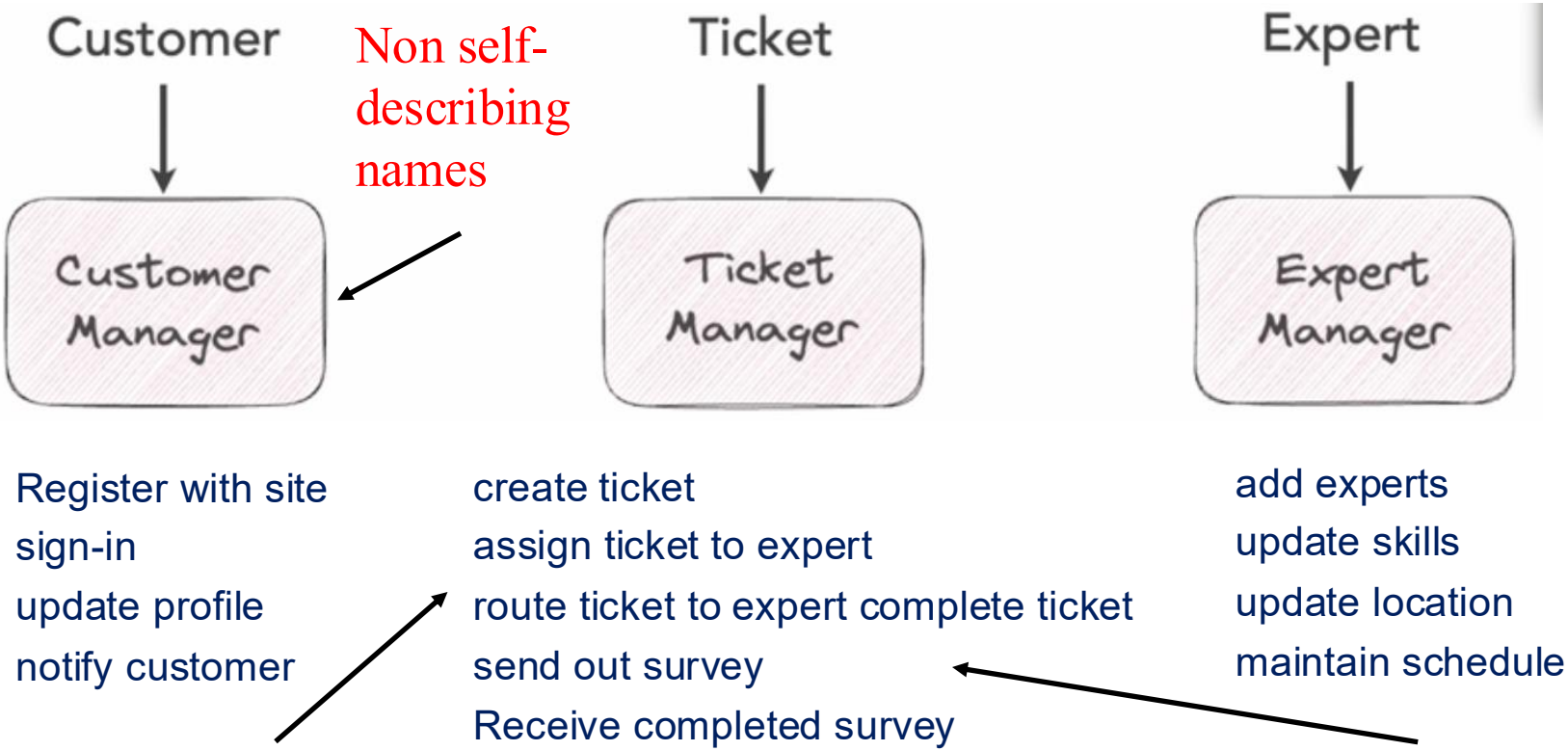
Customer Manager	Ticket Manager	Expert Manager
Register with site	create ticket	add experts
sign-in	assign ticket to expert	update skills
update profile	route ticket to expert	update location
notify customer	complete ticket	maintain schedule
	send out survey	
	Receive completed survey	

4.3. Component-based thinking



Discovering components

- **Entity trap anti-pattern**

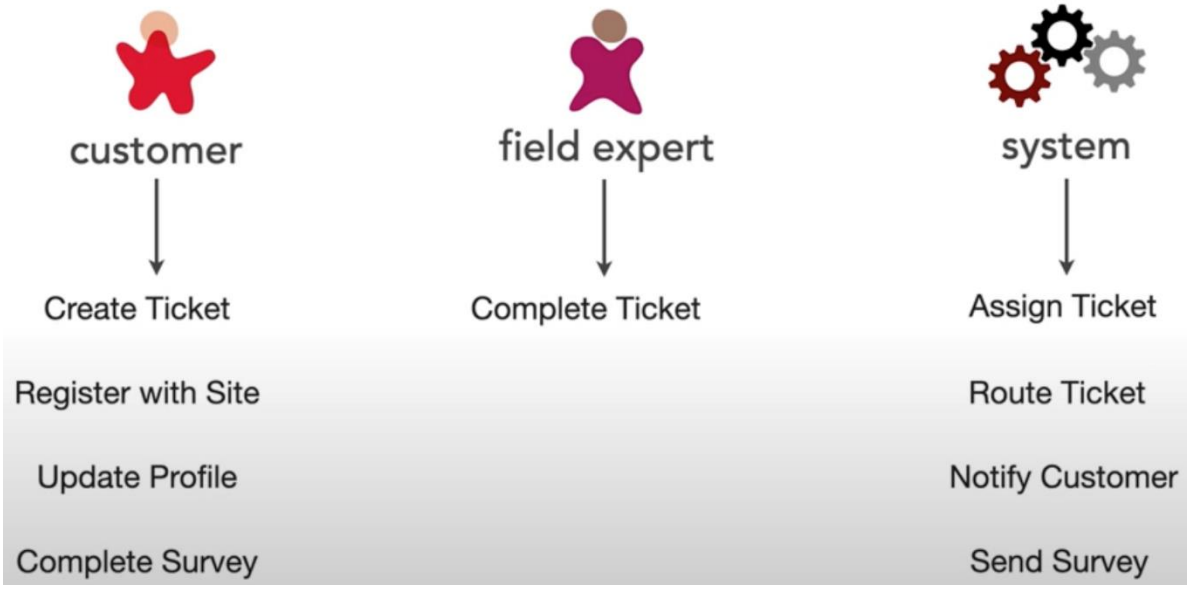


4.3. Component-based thinking



Discovering components

- **Actor/Actions approach**
  - The actor/actions approach is a popular way that architects use to map requirements to components.
  - In this approach, originally defined by the Rational Unified Process, architects identify actors who perform activities with the application and the actions those actors may perform.

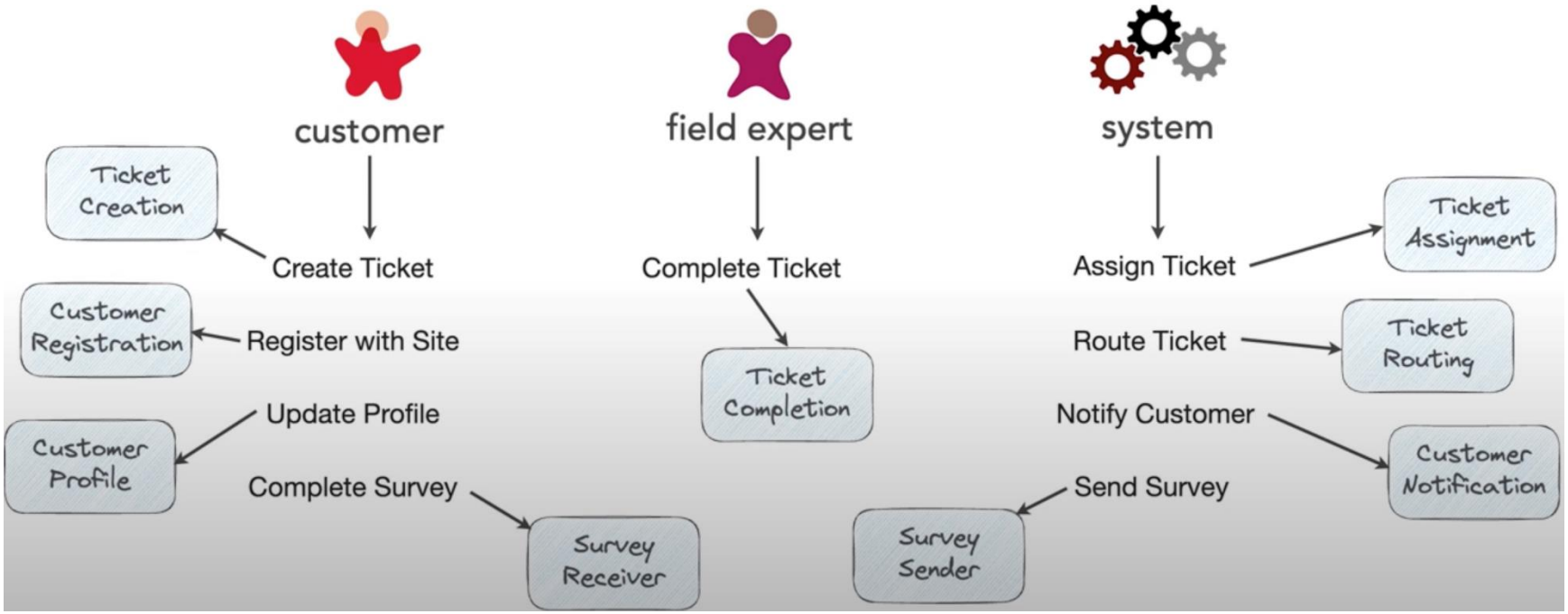


4.3. Component-based thinking



Discovering components

- **Actor/Actions approach**

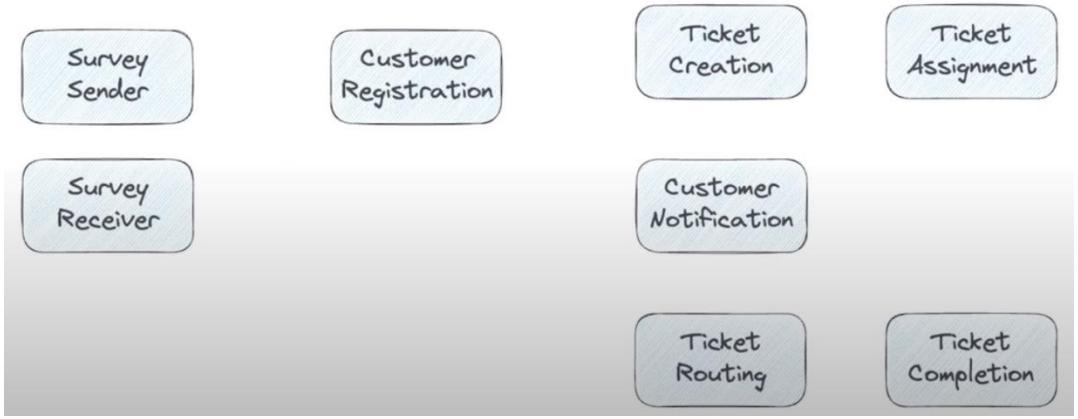


4.3. Component-based thinking



Discovering components

- **Actor/Actions approach**

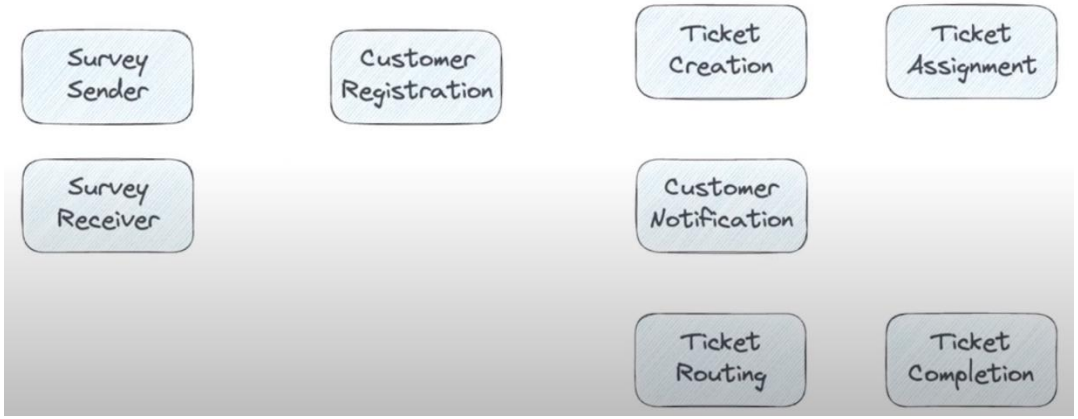


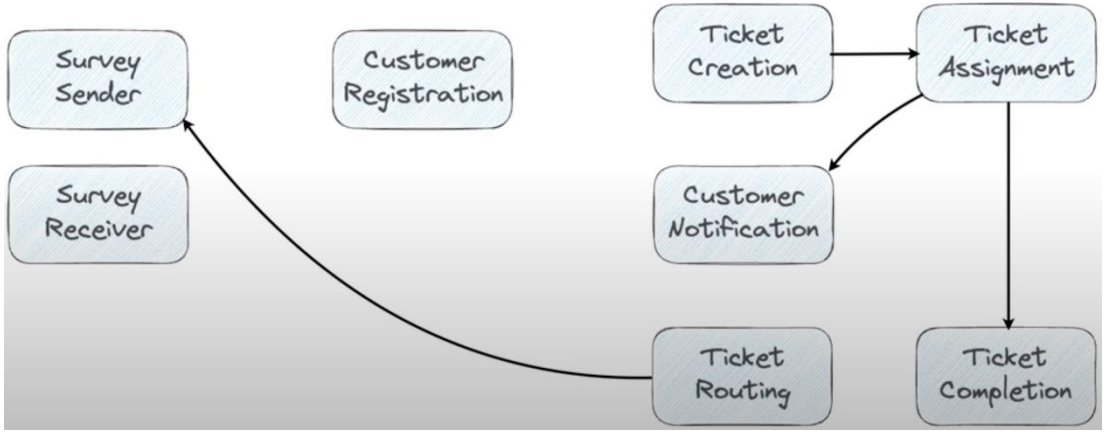
4.3. Component-based thinking



Discovering components

- **Actor/Actions approach**



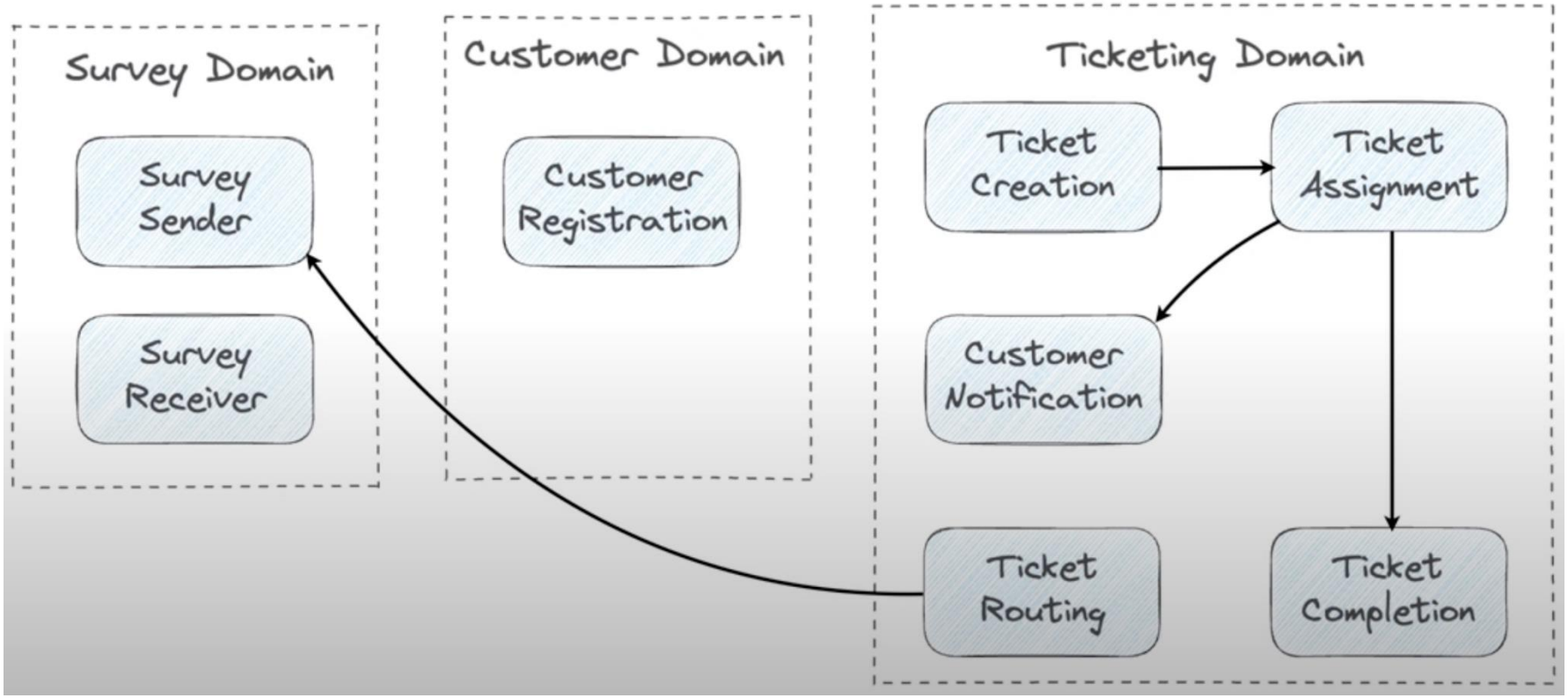


4.3. Component-based thinking



Discovering components

- **Actor/Actions approach**



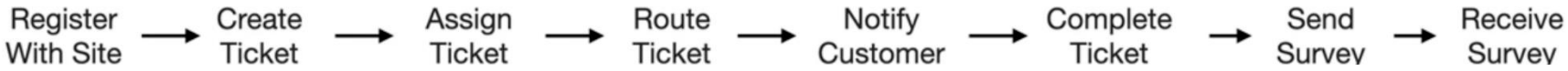
4.3. Component-based thinking



Discovering components

- **Workflow approach**

  Models the components around workflows. For examples:



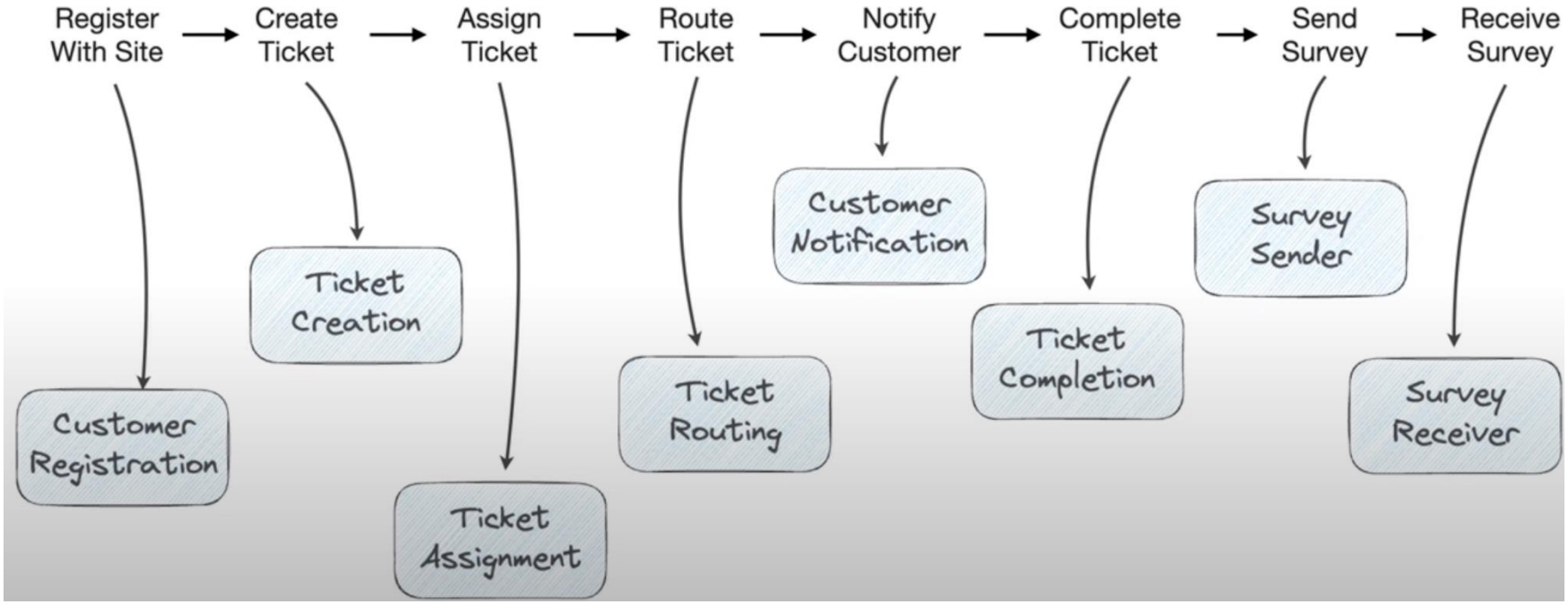
4.3. Component-based thinking



Discovering components

- **Workflow approach**

  Models the components around workflows. For examples:

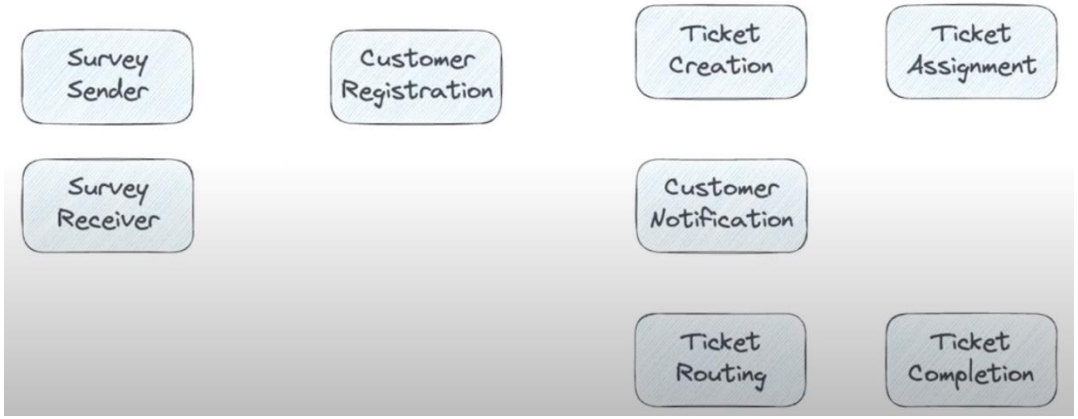


4.3. Component-based thinking



Discovering components

- **Workflow approach**

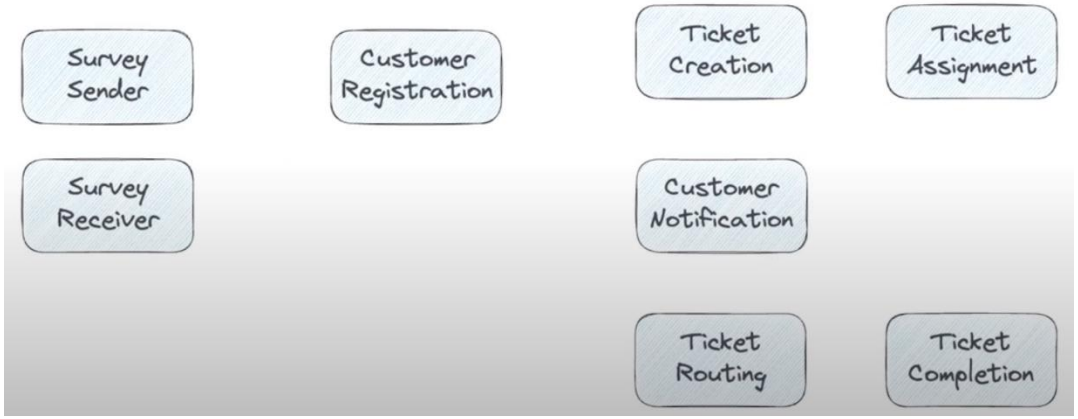


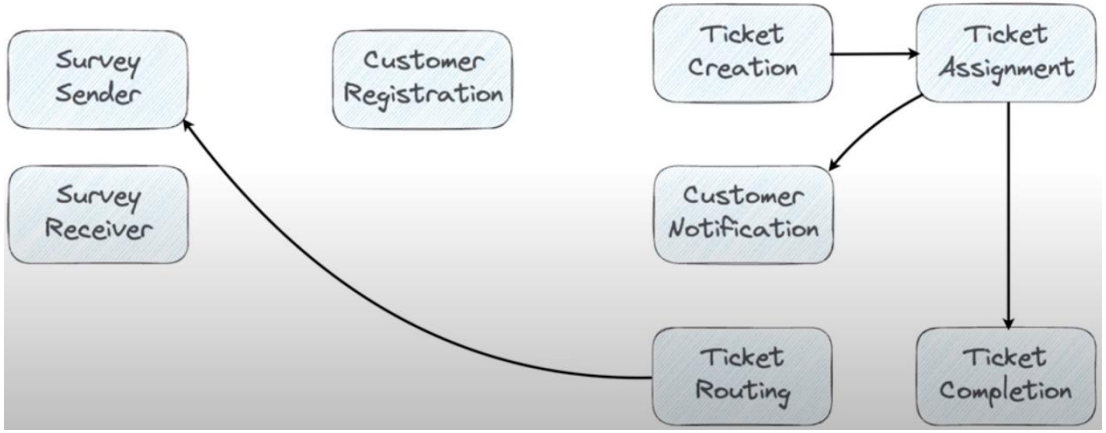
4.3. Component-based thinking



Discovering components

- **Workflow approach**



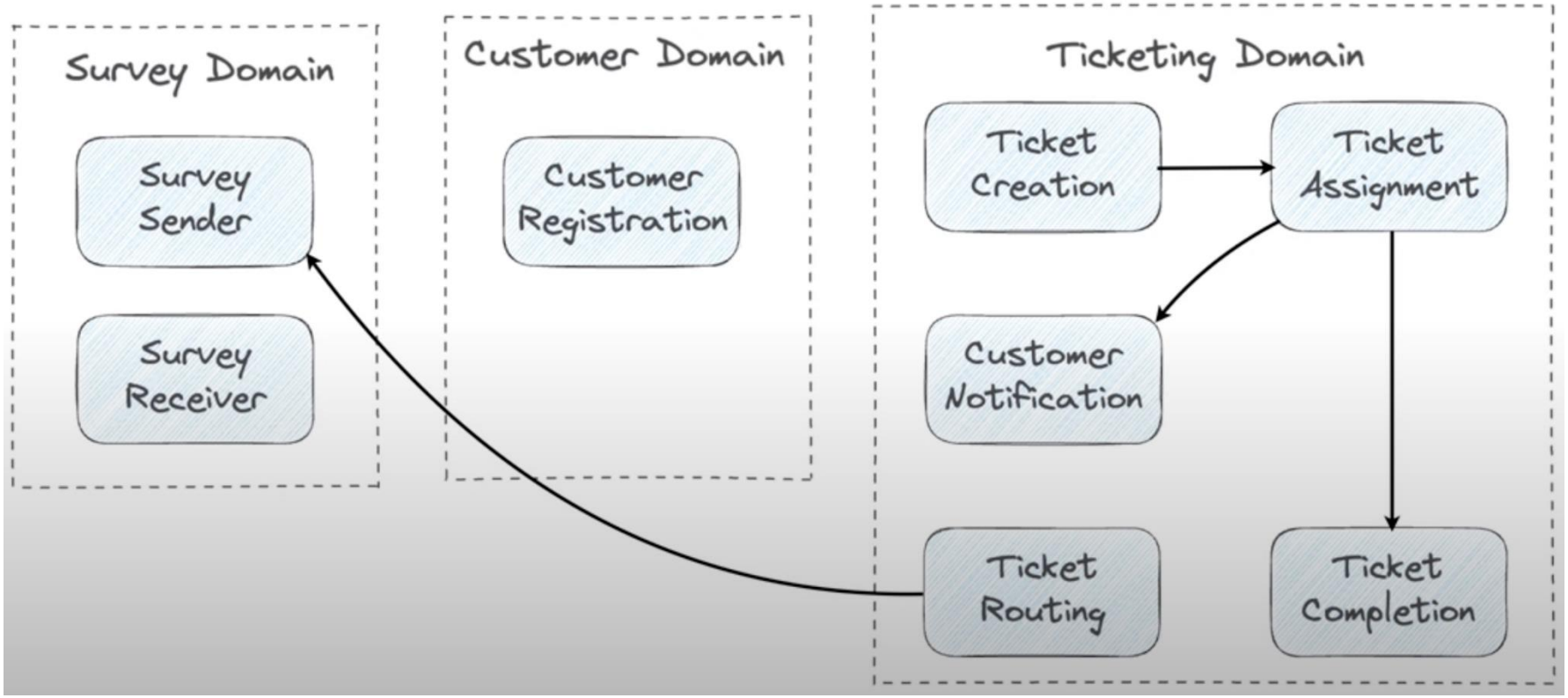


4.3. Component-based thinking



Discovering components

- **Workflow approach**



4.3. Component-based thinking



Exercise

An auction company wants to take their auctions online to a nationwide scale--customers choose the auction to participate in, wait until the auction begins, then bid during the live auction as if they were there in the room, with the auctioneer.

- **Users:** scale up to hundreds of participants (per auction), potentially up to thousands of participants, and as many simultaneous auctions as possible
- **Requirements:**
  - bidders can see a live video stream of the auction and see all bids as they occur
  - auctions must be as real-time as possible
  - both online and live bids must be received in the order in which they are placed
  - bidders register with credit card; system automatically charges card if bidder wins participants must be tracked via a reputation index
- **Additional Context:**
  - auction company is expanding aggressively by merging with smaller competitors
  - if nationwide auction is a success, replicate the model overseas
  - budget is not constrained--this is a strategic direction
  - company just exited a lawsuit where they settled a suit alleging fraud

4.3. Component-based thinking



Architecture Quantum Redux: Choosing Between Monolithic Versus Distributed Architectures

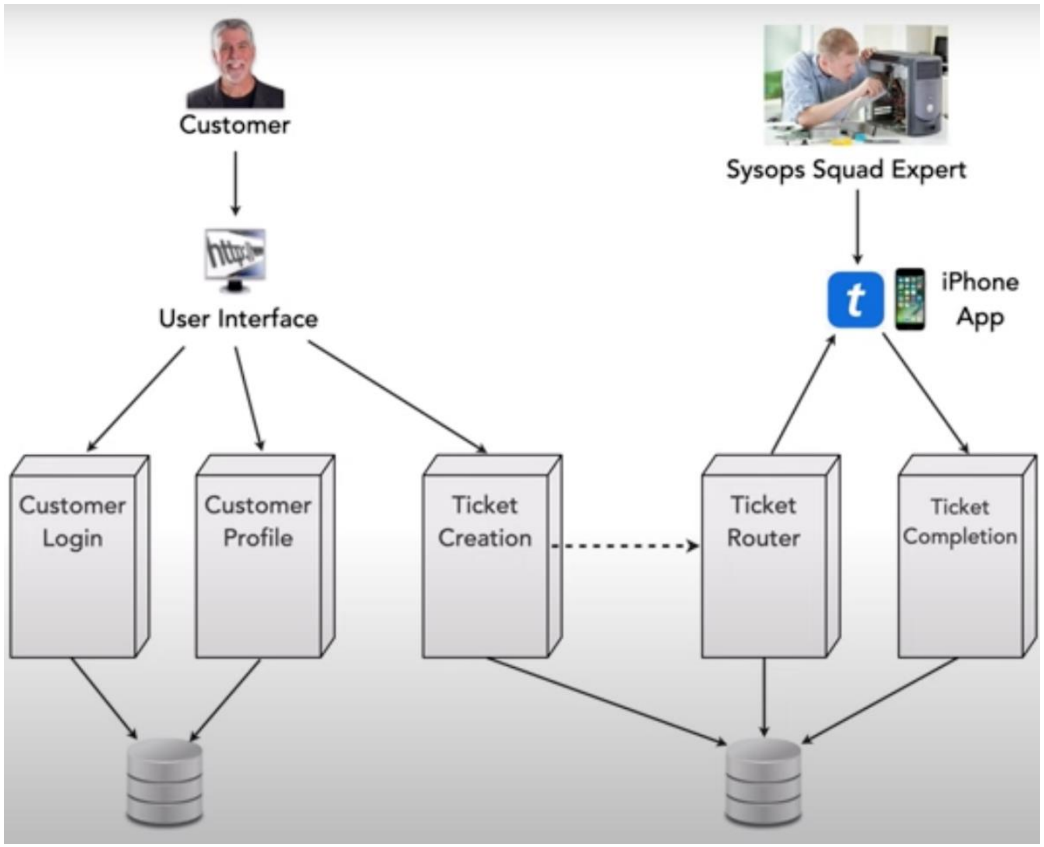
- Architecture quantum: an independently deployable artifact with high functional cohension and synchronous dynamic coupling.

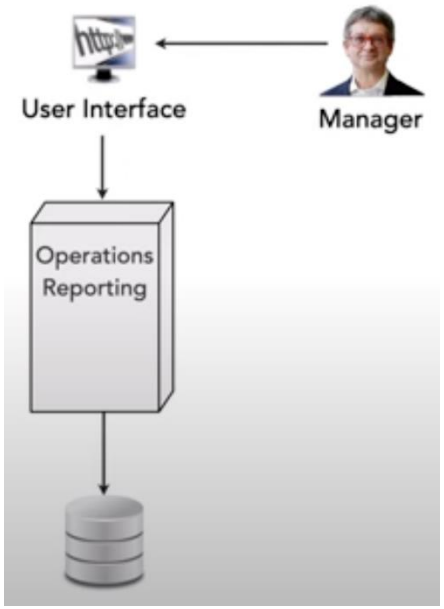
4.3. Component-based thinking



Architecture Quantum Redux: Choosing Between Monolithic Versus Distributed Architectures

- Senario:



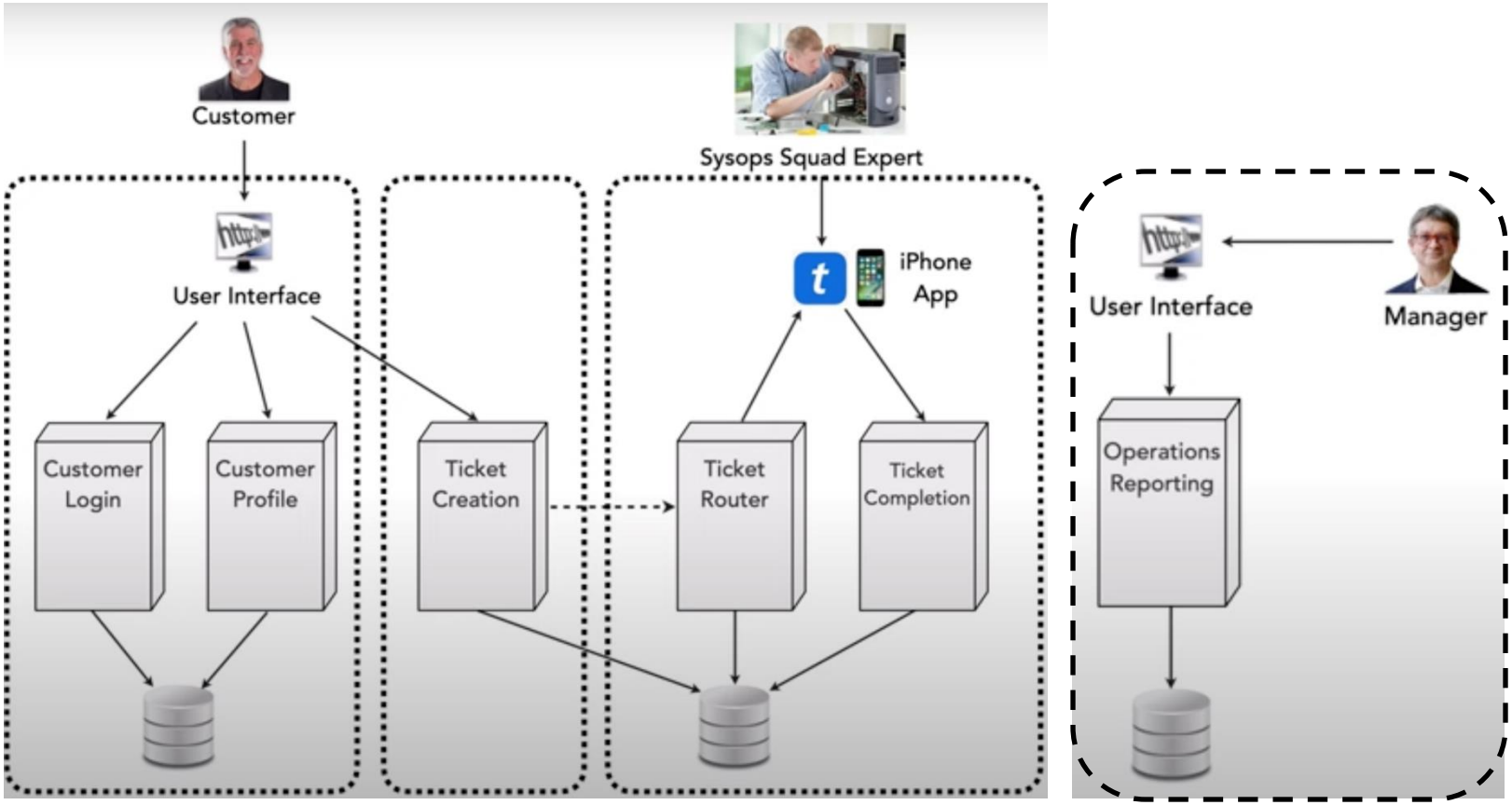


4.3. Component-based thinking



Architecture Quantum Redux: Choosing Between Monolithic Versus Distributed Architectures

- Senario:

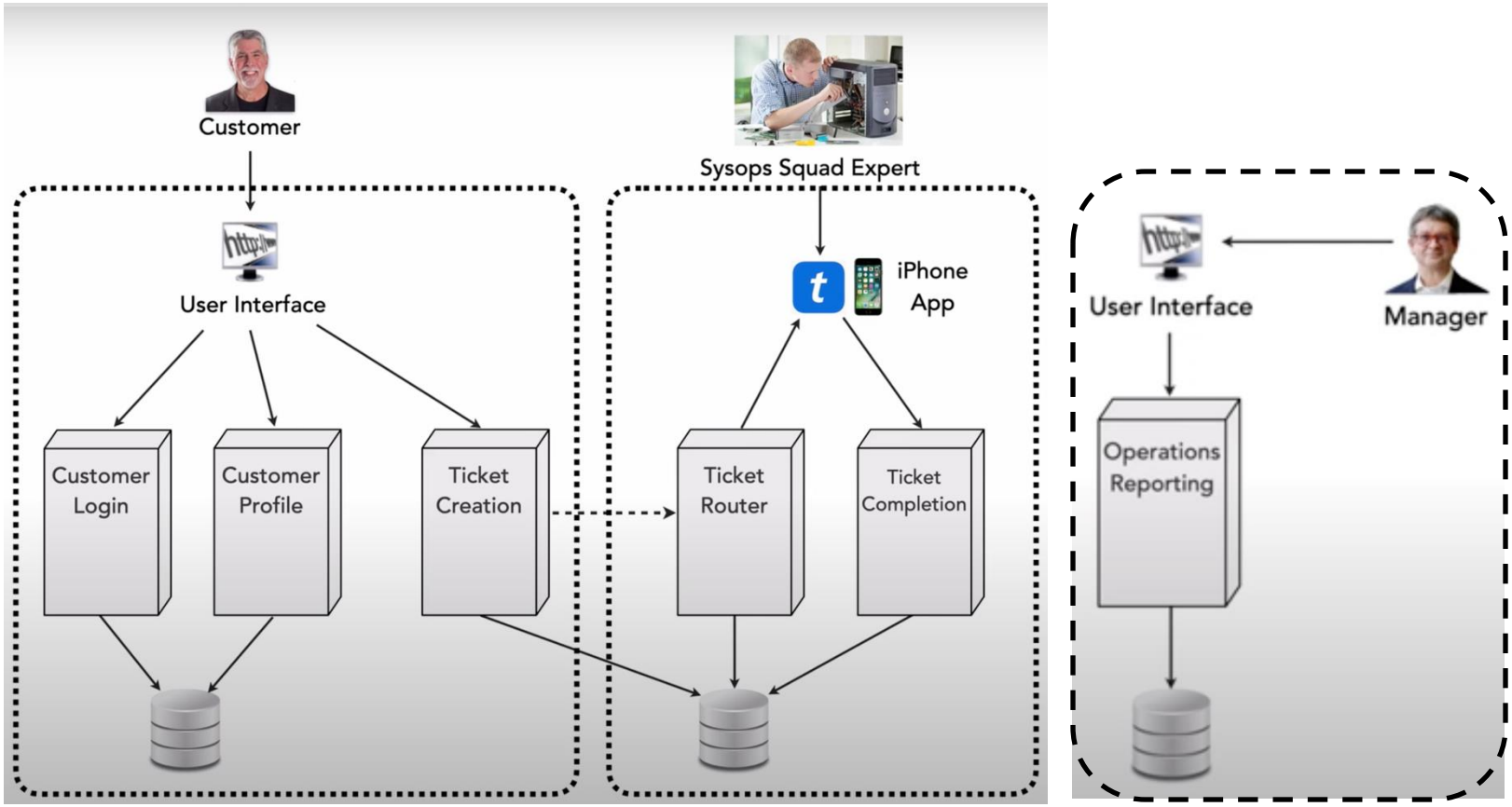


4.3. Component-based thinking



Architecture Quantum Redux: Choosing Between Monolithic Versus Distributed Architectures

- Senario:

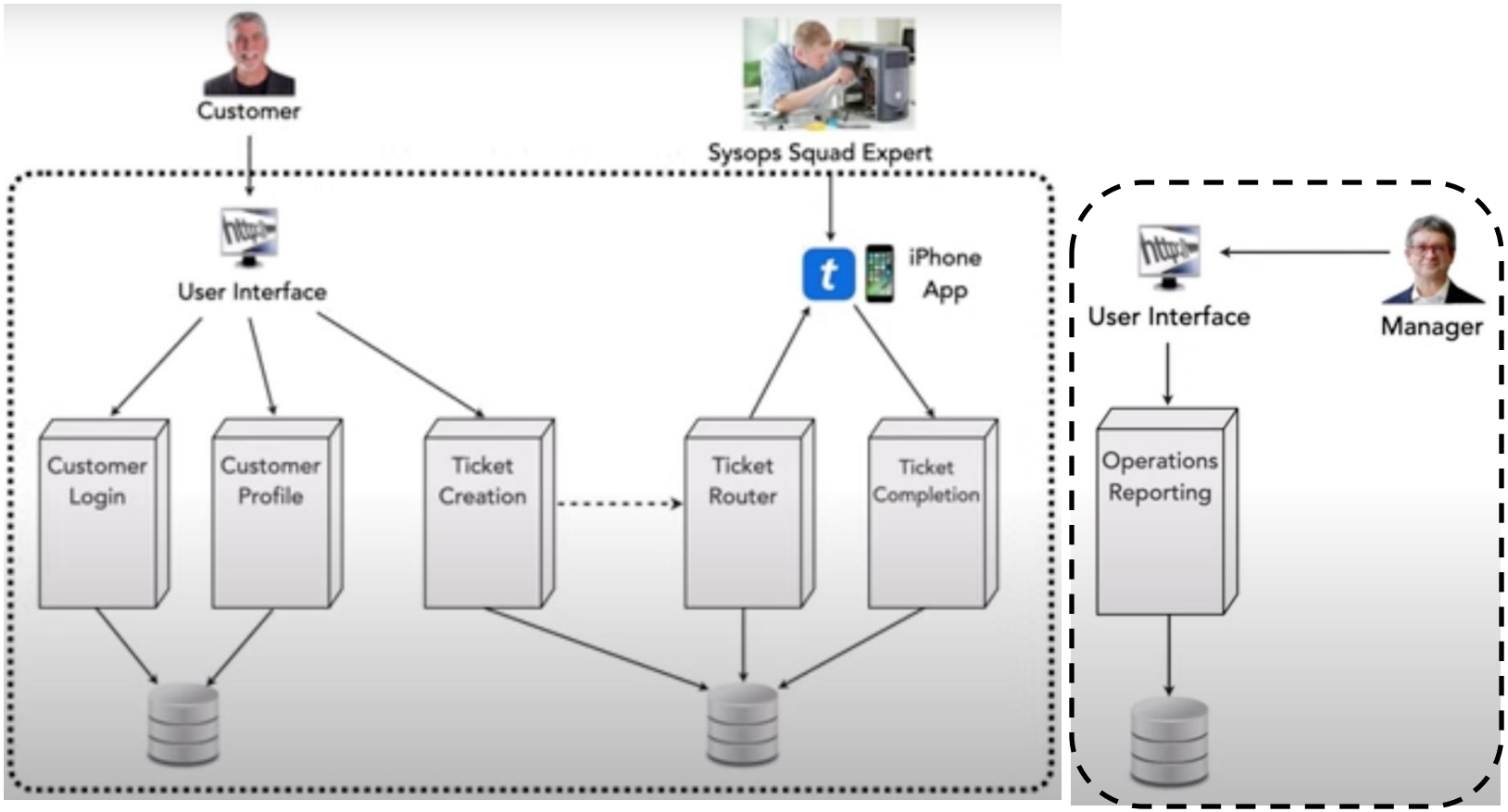


4.3. Component-based thinking



Architecture Quantum Redux: Choosing Between Monolithic Versus Distributed Architectures

- Senario:

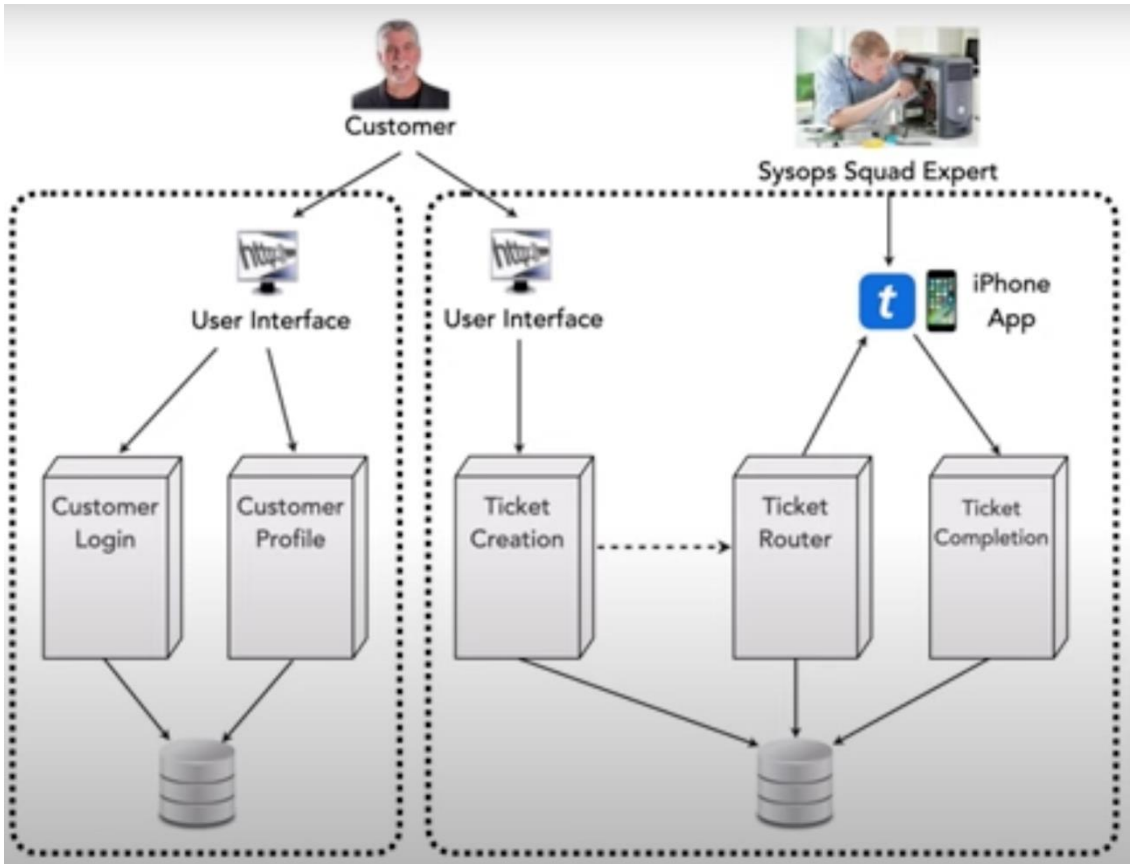


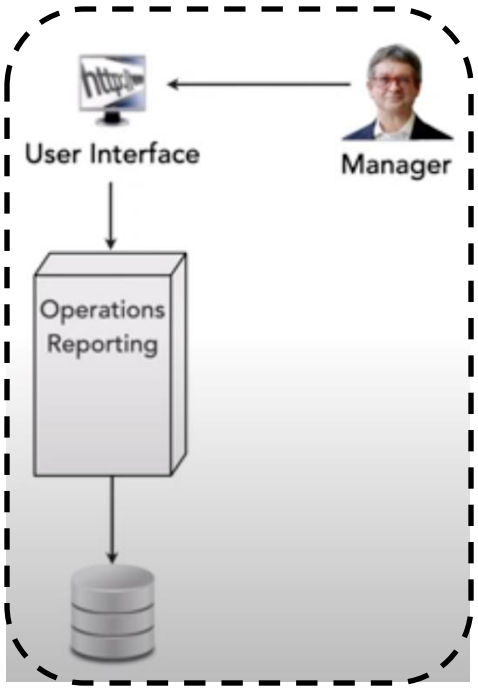
4.3. Component-based thinking



Architecture Quantum Redux: Choosing Between Monolithic Versus Distributed Architectures

- Senario:



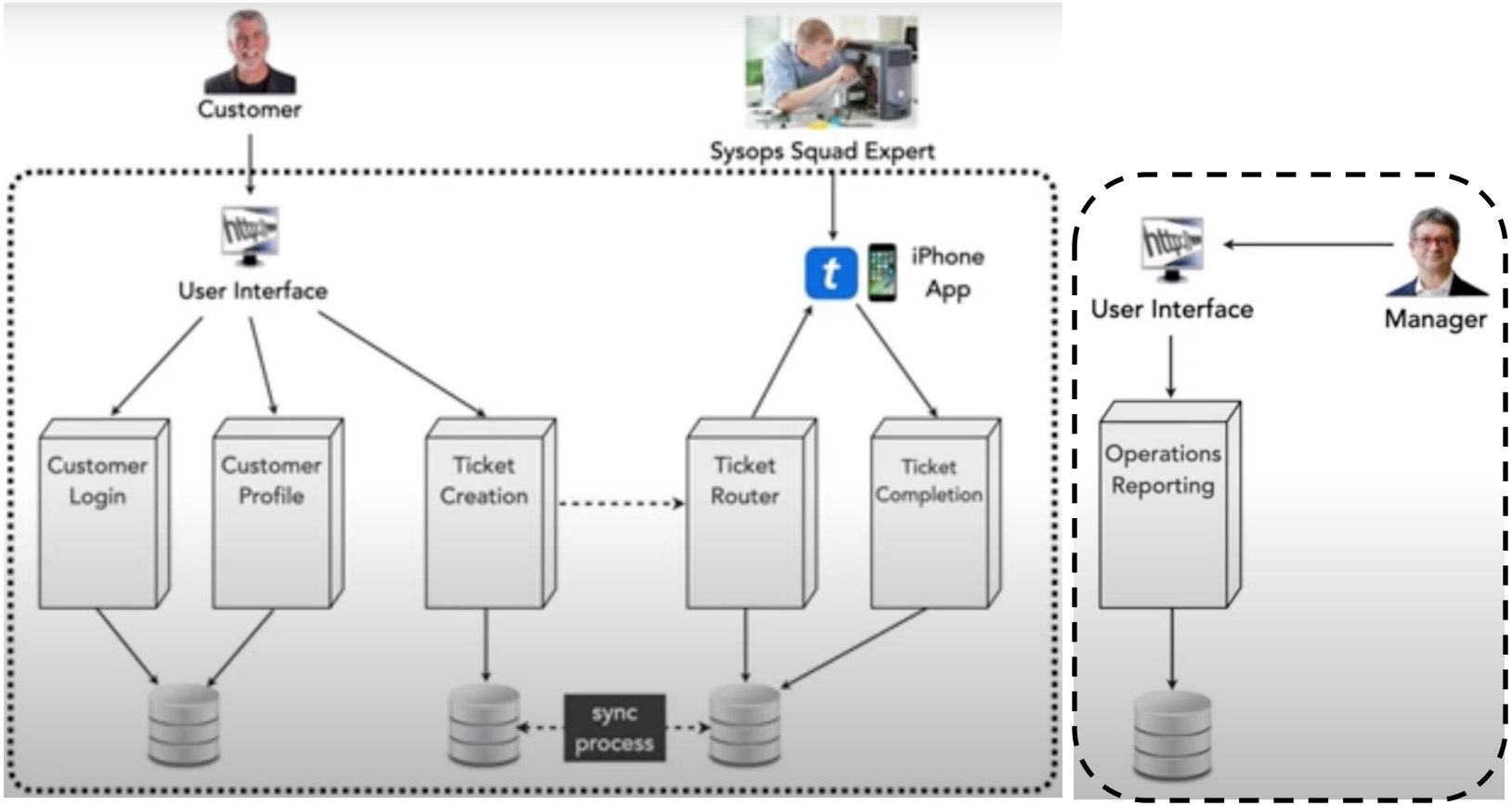


4.3. Component-based thinking



Architecture Quantum Redux: Choosing Between Monolithic Versus Distributed Architectures

- Senario:

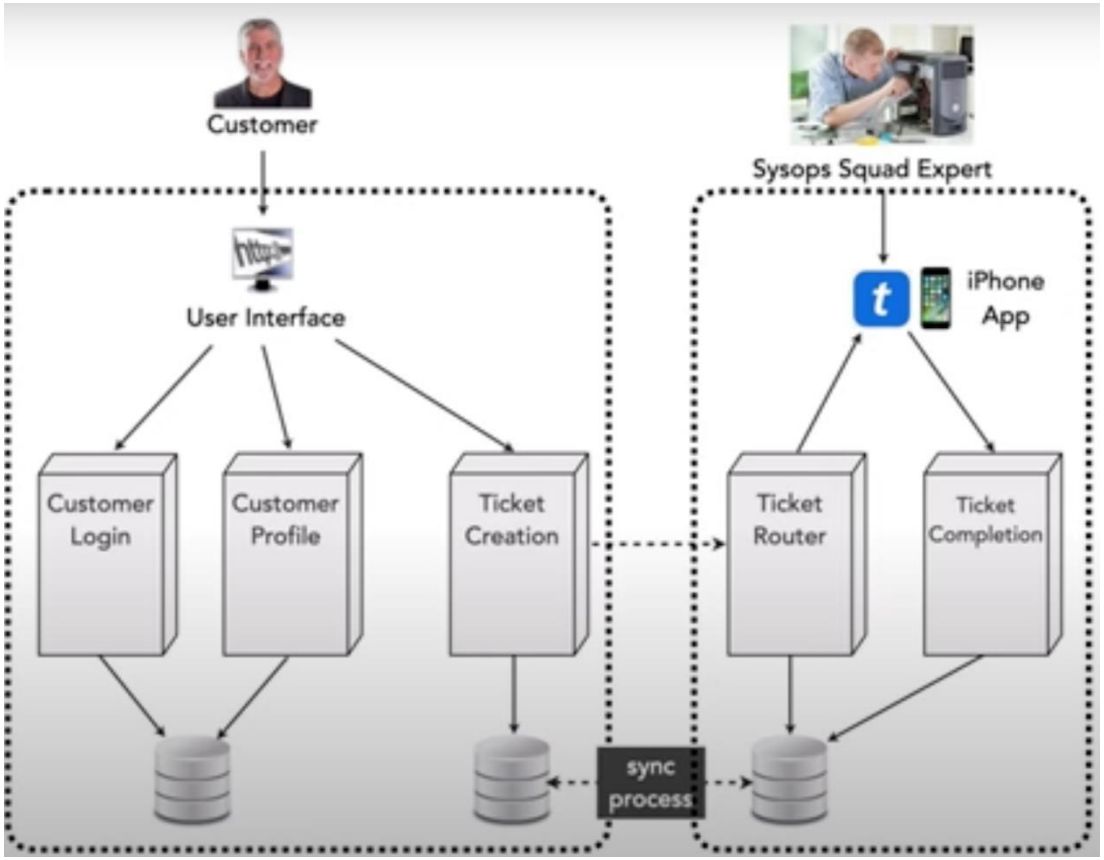


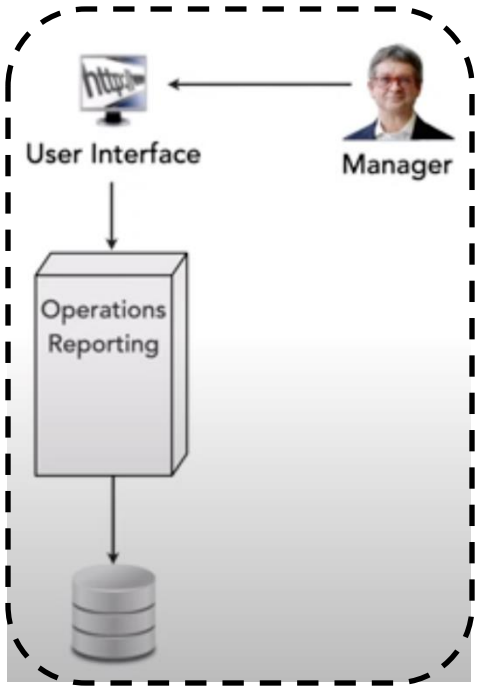
4.3. Component-based thinking



Architecture Quantum Redux: Choosing Between Monolithic Versus Distributed Architectures

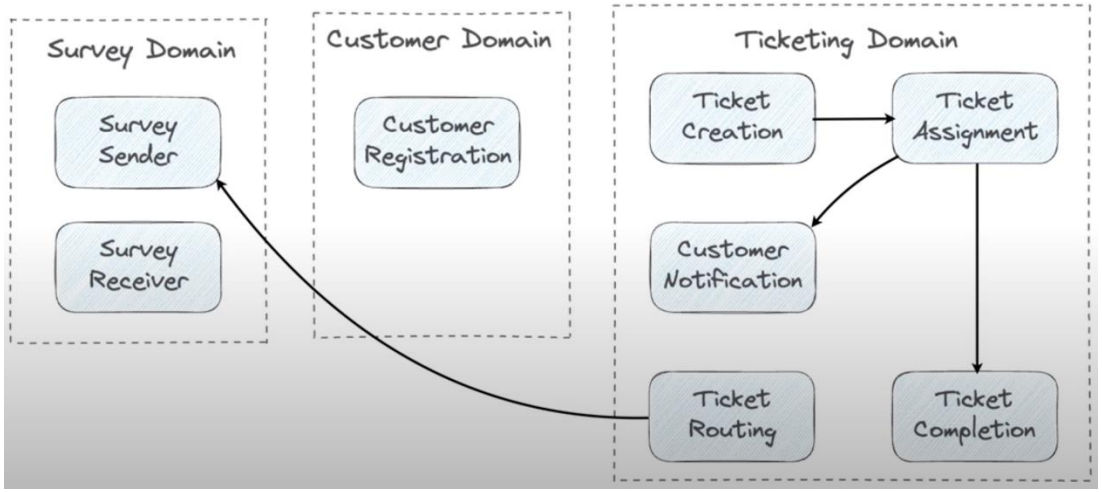
- Senario:





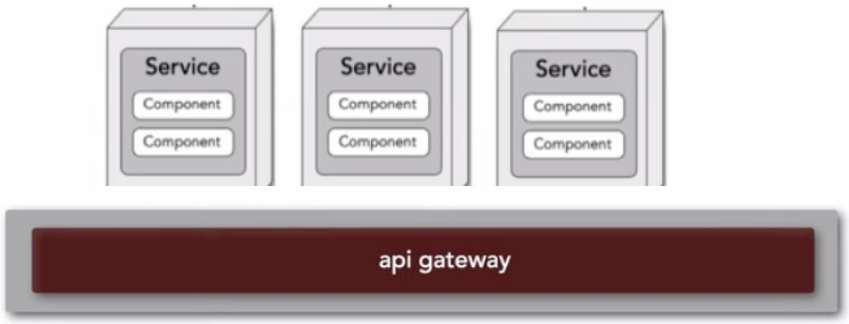
4.3. Component-based thinking

Logical vs Physical Architecture



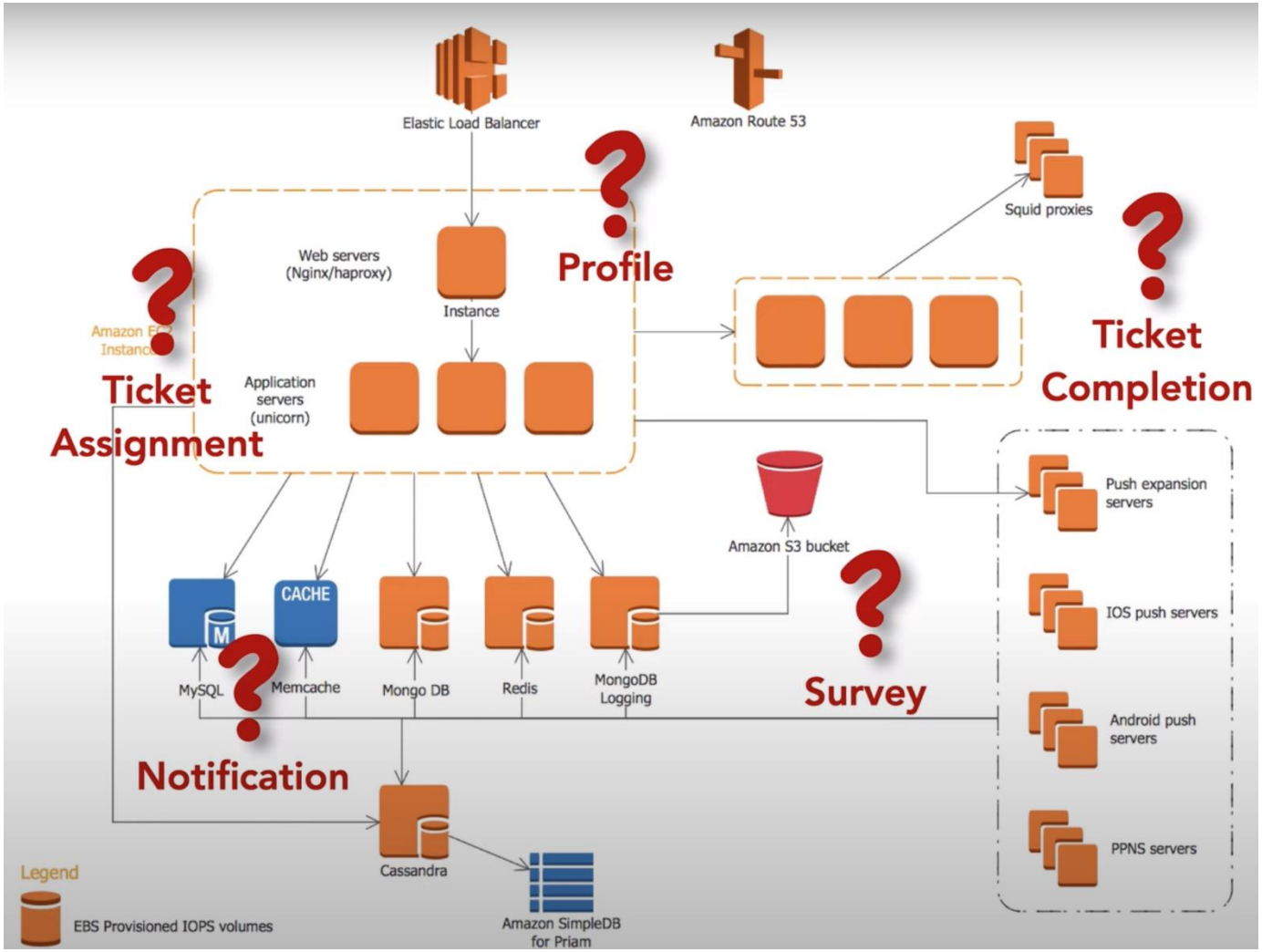
Mapping logical to physical architecture?





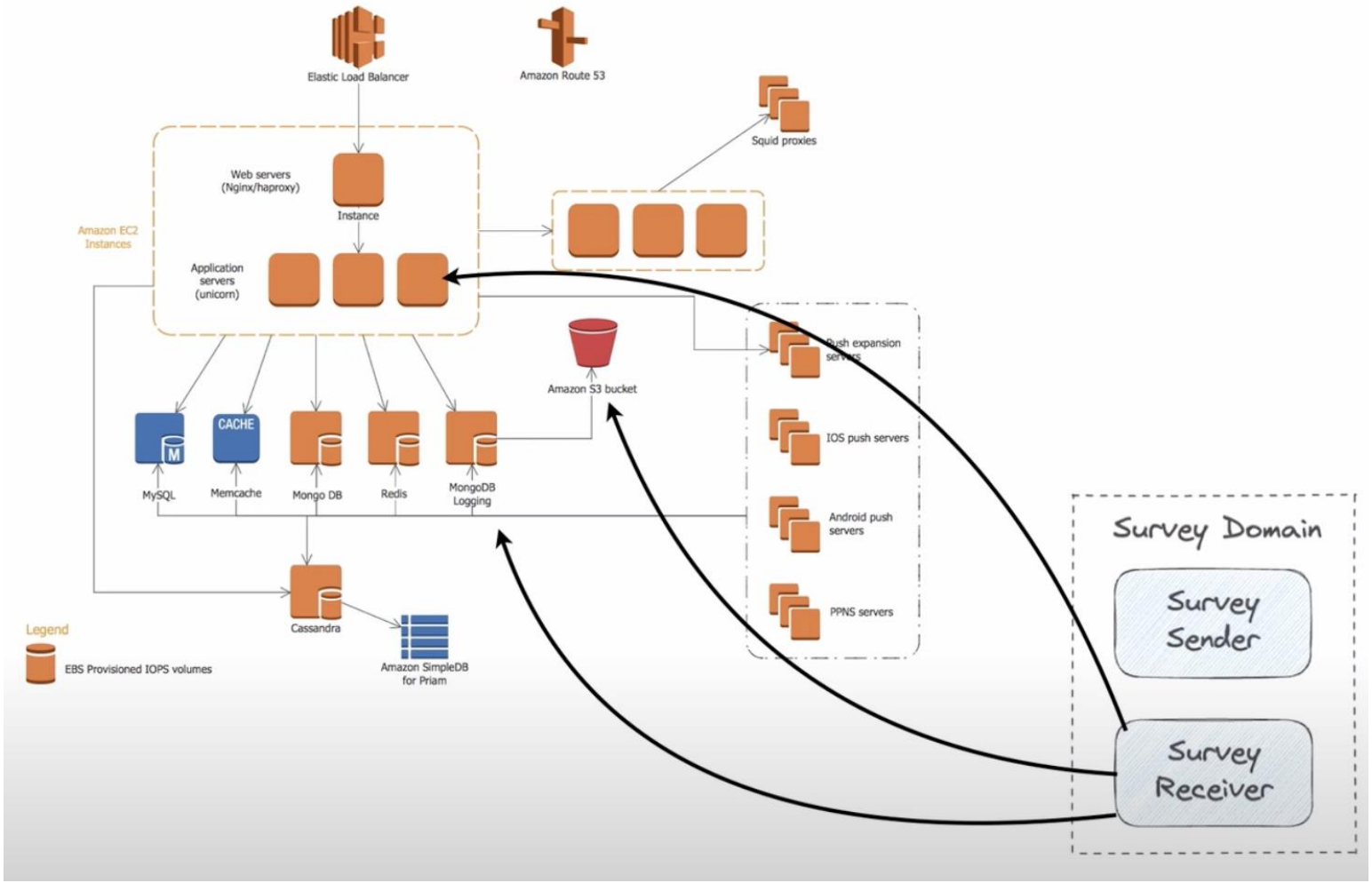
4.3. Component-based thinking

Logical vs Physical Architecture



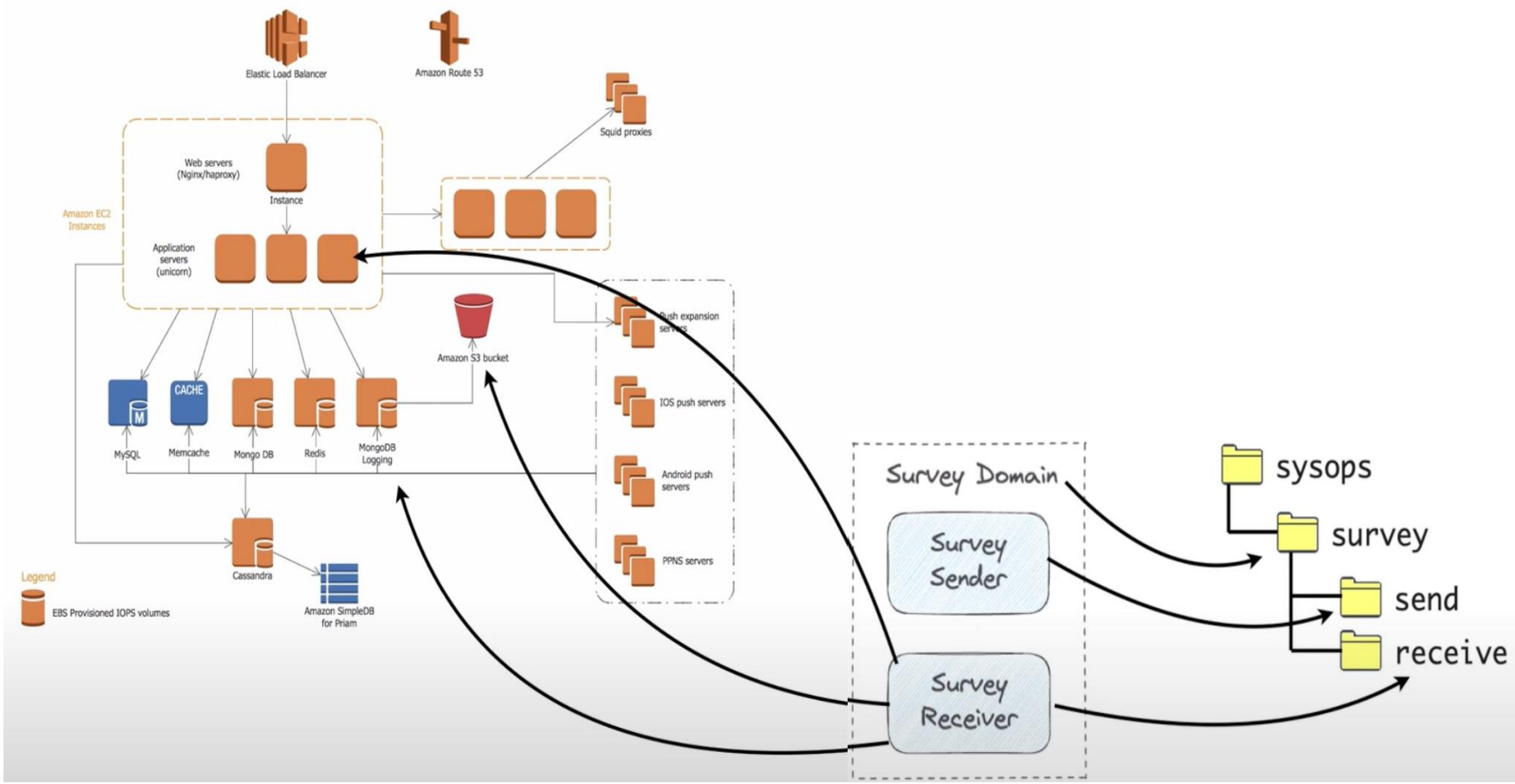
4.3. Component-based thinking

Logical vs Physical Architecture



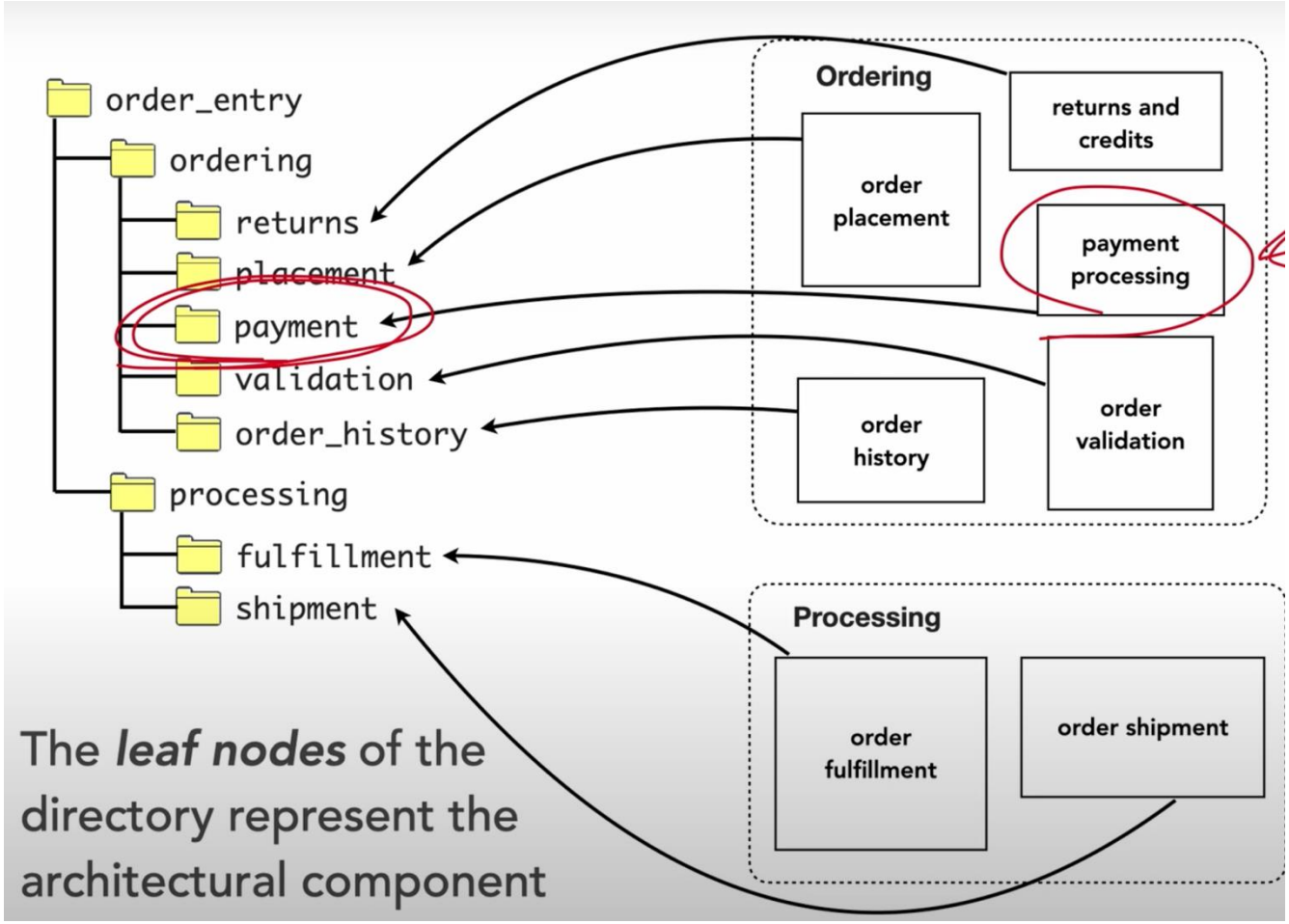
4.3. Component-based thinking

Logical vs Physical Architecture



4.3. Component-based thinking

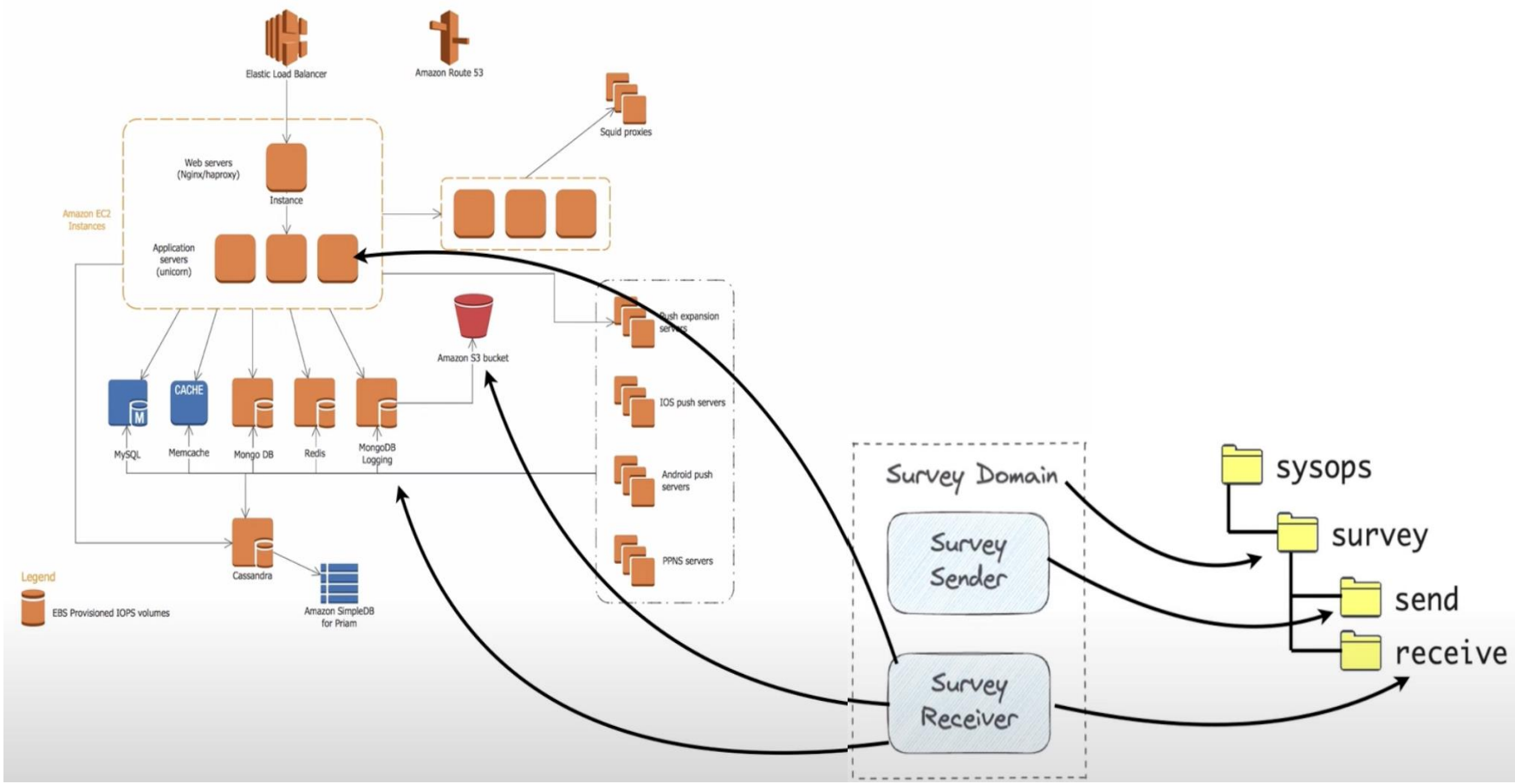
Logical vs Physical Architecture



4.3. Component-based thinking

Logical vs Physical Architecture

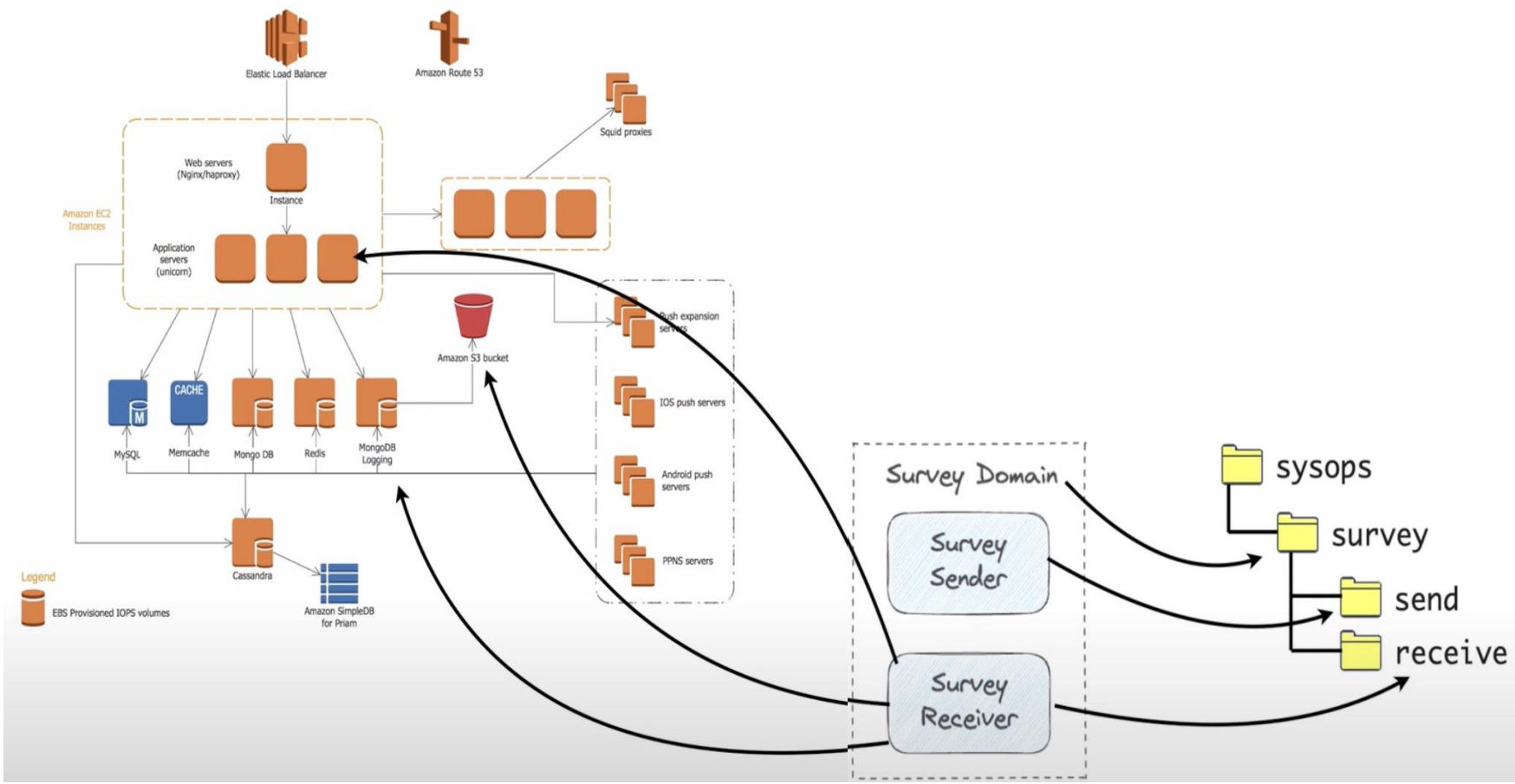
- But how?



4.3. Component-based thinking

Logical vs Physical Architecture

- But how? Choose an **architecture style**



4.3. Component-based thinking



Logical vs Physical Architecture

- But how? Choose an **architecture style**

Architecture Styles Worksheet

System/Project: _____

Architect/Team: _____ Date: _____

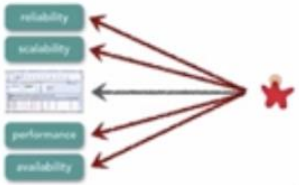
Selected Architecture(s):

	☐ layered	☐ modular monolith	☐ microkernel	☐ microservices	☐ service-based	☐ service-oriented	☐ event-driven	☐ space-based
partitioning	technical	domain	domain	domain	domain	technical	technical	technical
cost	$	$	$	$$$$$	$$	$$$$	$$$	$$$$
maintainability	★	★★	★★★	★★★★★	★★★★	★	★★★	★★★
testability	★★	★★	★★★	★★★★★	★★★★	★	★★	★
deployability	★	★★	★★★	★★★★★	★★★★	★	★★★	★★★
simplicity	★★★★★	★★★★★	★★★★	★	★★★	★	★★	★
scalability	★	★	★	★★★★★	★★★	★★★	★★★★	★★★★★
elasticity	★	★	★	★★★★	★★	★★★	★★★	★★★★★
responsiveness	★★★	★★★	★★★	★★	★★★	★★	★★★★★	★★★★★
fault-tolerance	★	★	★	★★★★★	★★★★	★★★	★★★★★	★★★
evolvability	★	★	★★★	★★★★★	★★★★	★	★★★★★	★★★
abstraction	★	★	★★★	★	★	★★★★★	★★★★	★
interoperability	★	★	★★★	★★★	★★	★★★★★	★★★	★★

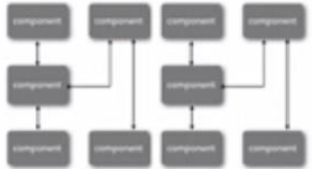
Conclusion



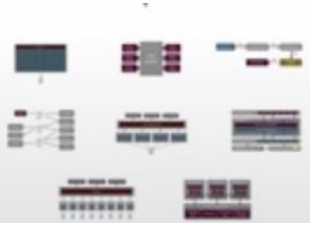
Problem description



Architecture characteristics



Logical architecture



Physical architecture



Documenting architecture



Presenting the solution



Q&A